Java Full Stack Development

1. Oracle Data Modeling

Module Topics

- Data Modeling
- Oracle Tables and Storage
- Data Types
- Keys
- Relationships
- Constraints

Data Modeling

Domain Modeling

- User's view of the data
  - Expressed in terms of objects
  - Discovered in domain analysis
  - Modeled by high level class diagrams
  - Main actions are often objects
    - "We need to talk about the relationship"
- Idiosyncratic
  - Each user or group has their own model
- Normally the responsibility of the BA

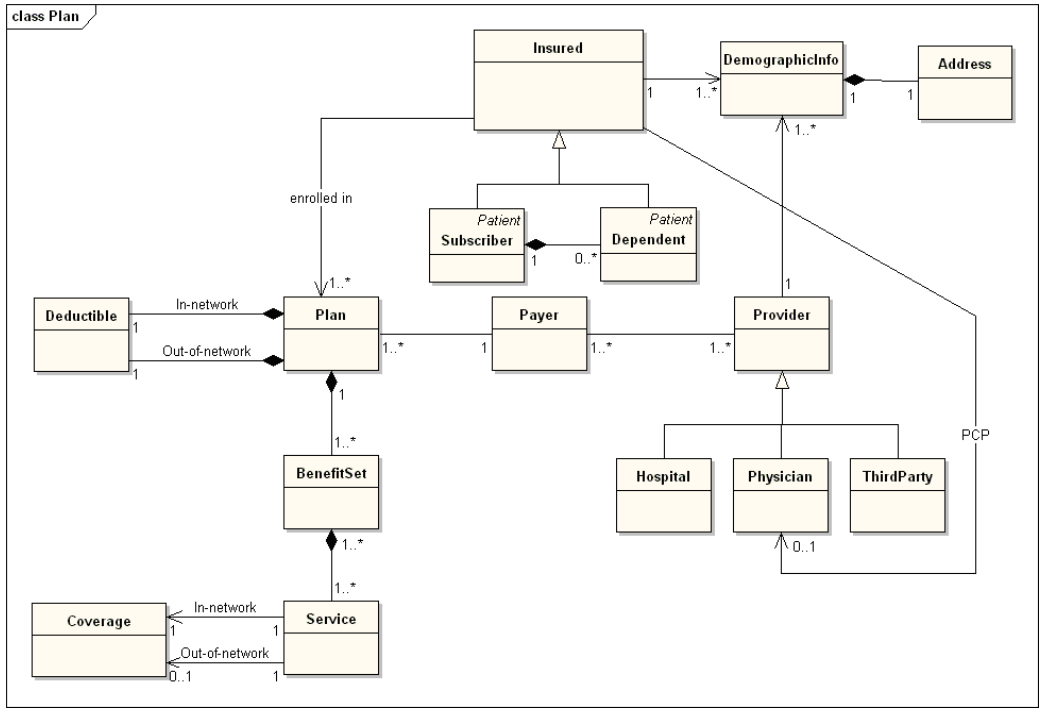


Image Credit: https://en.wikipedia.org/wiki/Domain_model

Domain Modeling

- Common mistake
  - Generating a relational model directly from a domain model
  - The domain model is generally not precise, accurate or robust enough to generate a solid relational model
  - Generating the relational model from a domain model
    - Means the database only represents one group of users' view of the data
    - Generally becomes brittle, complex and unworkable in production

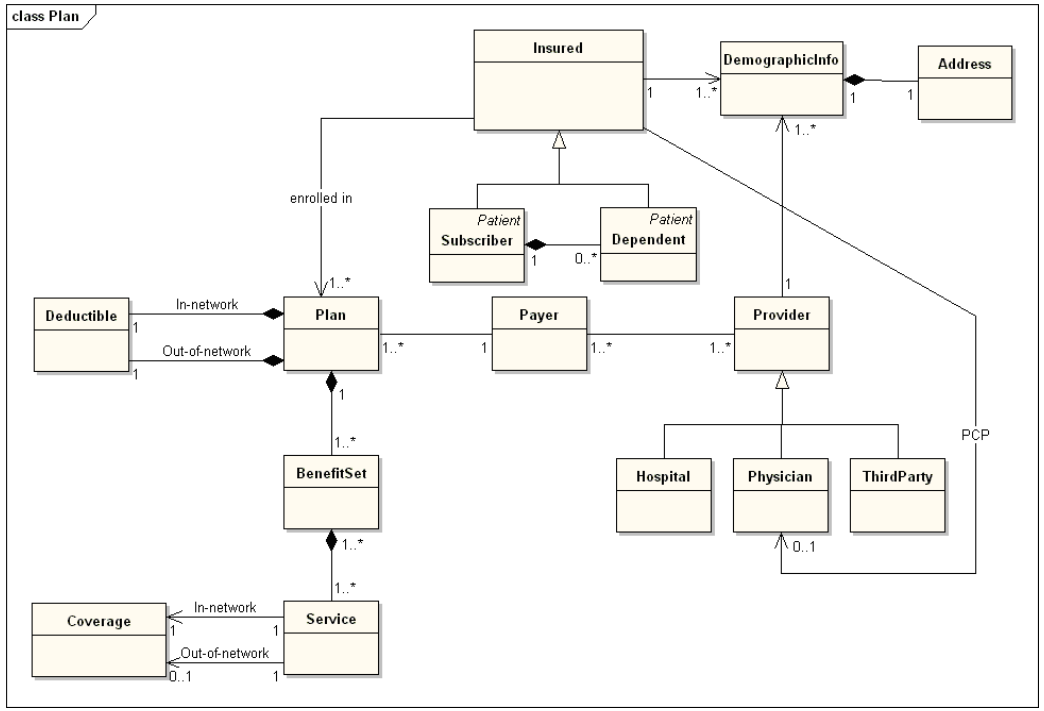


Image Credit: https://en.wikipedia.org/wiki/Domain_model

Chen and Codd

- E.F. Codd introduced the relational model in 1970
  - Based on mathematical set theory and first-order predicate logic
  - Defined how data should be stored and queried in a database system
  - The SQL language was built on this theoretical foundation

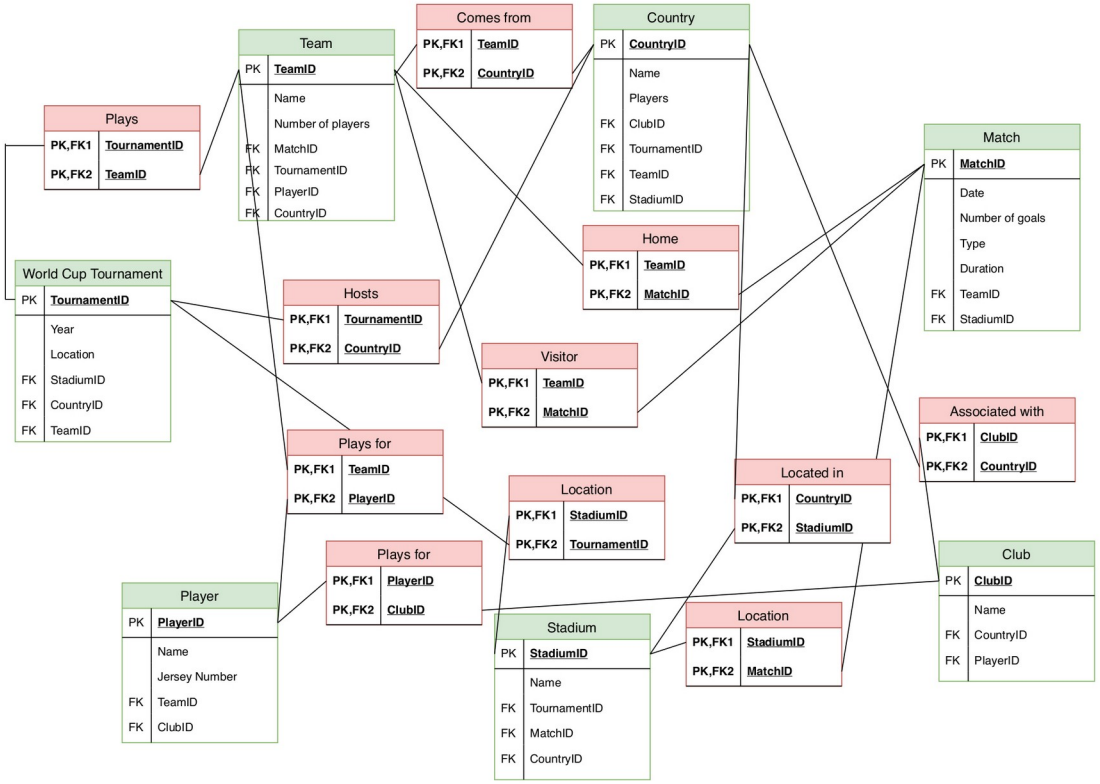
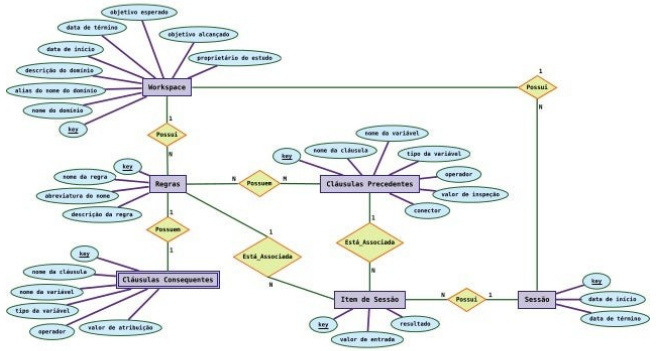


Image Credit: https://storage.googleapis.com/dajhwfuteugmme/what-is-data-entity-model.html

Chen and Codd

- Peter Chen introduced entity-relationship modeling in 1976
  - Defined how data should be discovered and described before implementation
  - Provided a way to model the real world independently of any storage technology
  - Introduced the four levels at which data can be understood

Entity-relationship Model



Relational Model

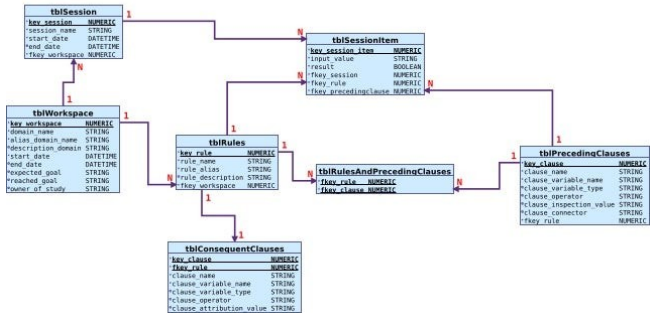
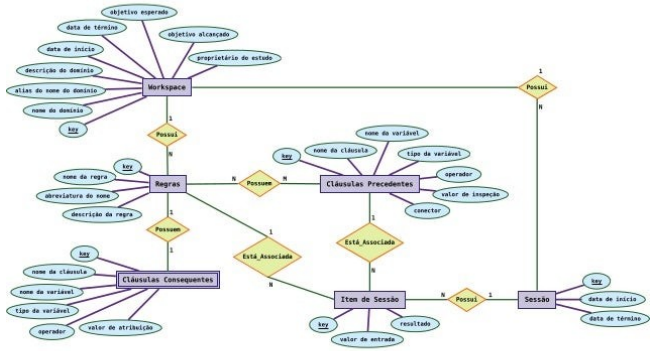


Image Credit: https://www.slideshare.net/michelalves/diagrama-entidaderelacionamento

Chen and Codd

- The two frameworks are complementary
  - Chen's framework
    - Guides the modeling process
    - How to move from domain understanding to a formal definition
  - Codd's relational model
    - Provides the implementation target
    - One possible physical realization of the logical model
  - Using both produces schemas that are both semantically correct and technically sound

Entity-relationship Model



Relational Model

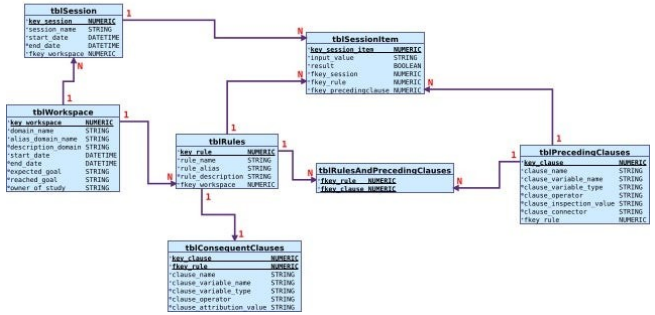
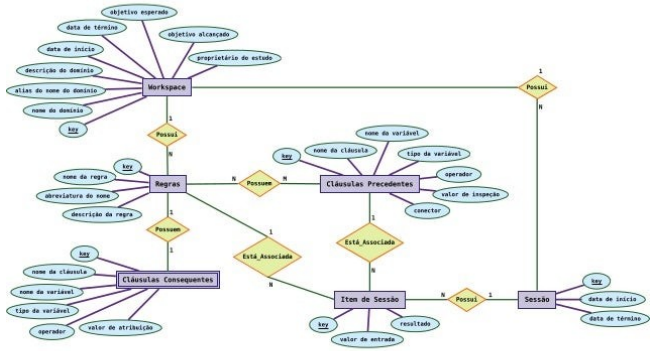


Image Credit: https://www.slideshare.net/michelalves/diagrama-entidaderelacionamento

Chen and Codd

- Modeling is not the same as implementation
  - Designing a data model is an act of understanding and communication
  - Implementing a schema is an act of engineering
  - Conflating the two collapses a multi-stage process into a single step creates an unstable data environment
  - The same logical model can be in multiple implementation models
  - For example
    - A logical model can be implemented in a dimensional model for OLAP processing

Entity-relationship Model



Relational Model

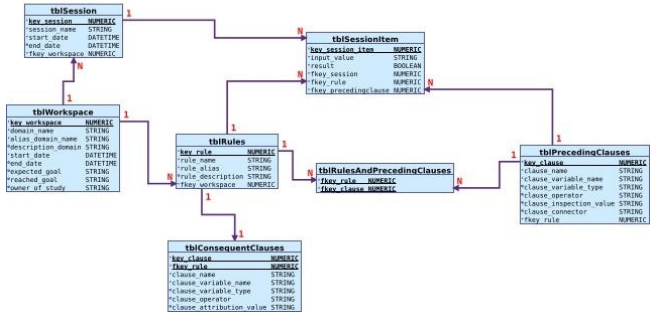


Image Credit: https://www.slideshare.net/michelalves/diagrama-entidaderelacionamento

Four Levels of Data

- Chen identified four distinct levels at which data must be understood
  - Each level addresses a different concern and involves different design decisions
  - The levels form a progression
    - Understanding precedes structure
    - Structure precedes access
  - Each level uses different modeling and design tools and logic

Level	Name	Description
1	Information about entities and relationships	Real-world things and their associations as they exist in the mind -- concepts, not data
2	Information structure	How those concepts are organized and represented as data: entities, attributes, relationships, and value sets
3	Access-path-independent data structure	A formal data structure, such as a relational table, that does not specify how data will be physically located or retrieved
4	Access-path-dependent data structure	The physical storage and retrieval structures: indexes, data structure diagrams, storage organization

Four Levels of Data

- Data modeling process flow
  - Discover what exists in the domain (Level 1) before attempting to define data structures
    - Multiple views of the domain
    - Inconsistent and incomplete
  - Define the information structure (Level 2) before choosing a data model
    - Strict definitions using predicates
    - Domain specific language (DSL)
  - Main functions of leve2 modeling
    - Remove inconsistencies from domain models
    - Create an standard set of definitions that all the stakeholders agree on
    - Identify the type of data to be used

Level	Name	Description
1	Information about entities and relationships	Real-world things and their associations as they exist in the mind -- concepts, not data
2	Information structure	How those concepts are organized and represented as data: entities, attributes, relationships, and value sets
3	Access-path-independent data structure	A formal data structure, such as a relational table, that does not specify how data will be physically located or retrieved
4	Access-path-dependent data structure	The physical storage and retrieval structures: indexes, data structure diagrams, storage organization

Four Levels of Data

- Data modeling process flow
  - Choose an access-path-independent structure (Level 3) before optimizing for access paths
    - This means selecting a model without being concerned about it is implemented
    - For example; relational model, document model (MongoDB), columnar versus row oriented storage
  - Define physical access structures (Level 4) last, driven by performance requirements
    - Can be though of as the actual implementation of the model selected at level three
  - Level 4 issues include
    - How is the data physically organized in storage
    - How the software execute a join in SQL

Level	Name	Description
1	Information about entities and relationships	Real-world things and their associations as they exist in the mind -- concepts, not data
2	Information structure	How those concepts are organized and represented as data: entities, attributes, relationships, and value sets
3	Access-path-independent data structure	A formal data structure, such as a relational table, that does not specify how data will be physically located or retrieved
4	Access-path-dependent data structure	The physical storage and retrieval structures: indexes, data structure diagrams, storage organization

Four Levels of Data

- For relational data
  - Level three is declarative
    - We specify table and data formats
    - We use SQL to specify operations
  - Level four is imperative
    - How table data is actually moved in and out of persistent storage
    - May involve complex planning and optimization
  - For specific database product like Oracle
    - The level three model is like an interface
    - The level four product is the implementation of that interface

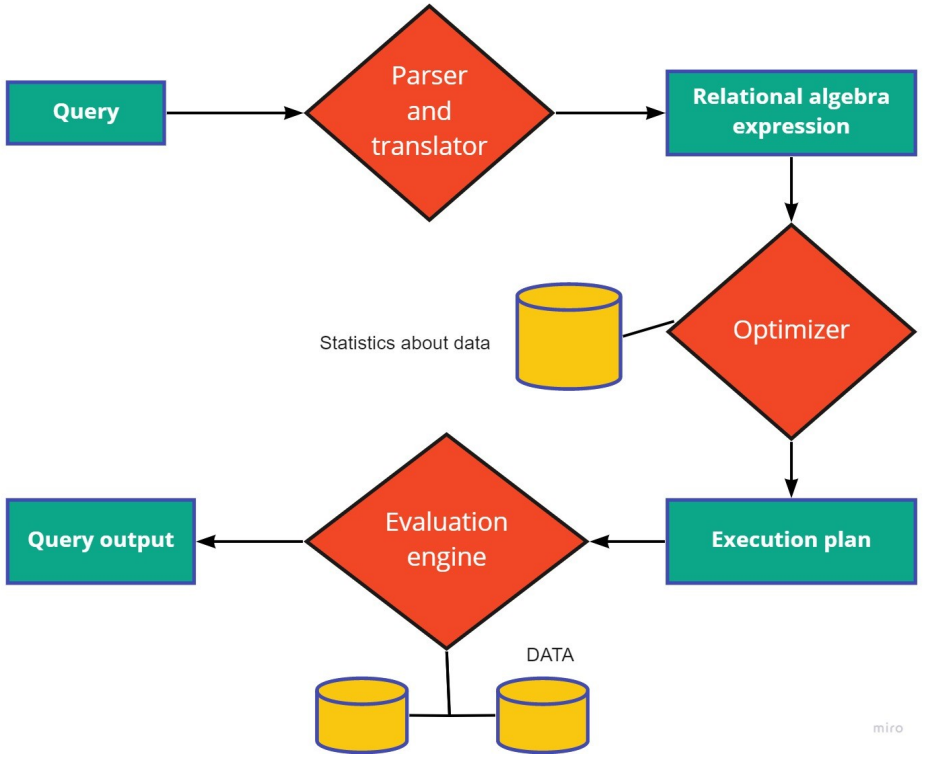
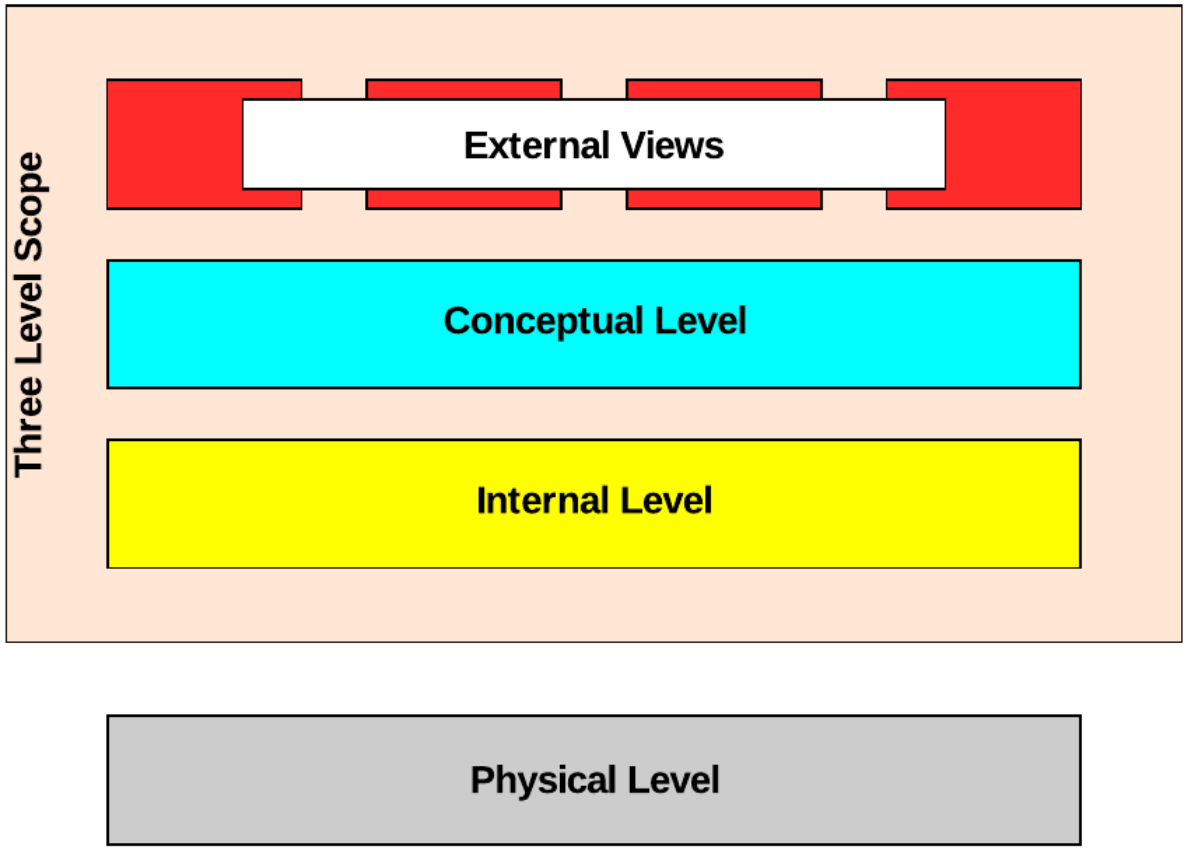


Image Credit: https://www.analyticsvidhya.com/blog/2021/10/a-detailed-guide-on-sql-query-optimization/

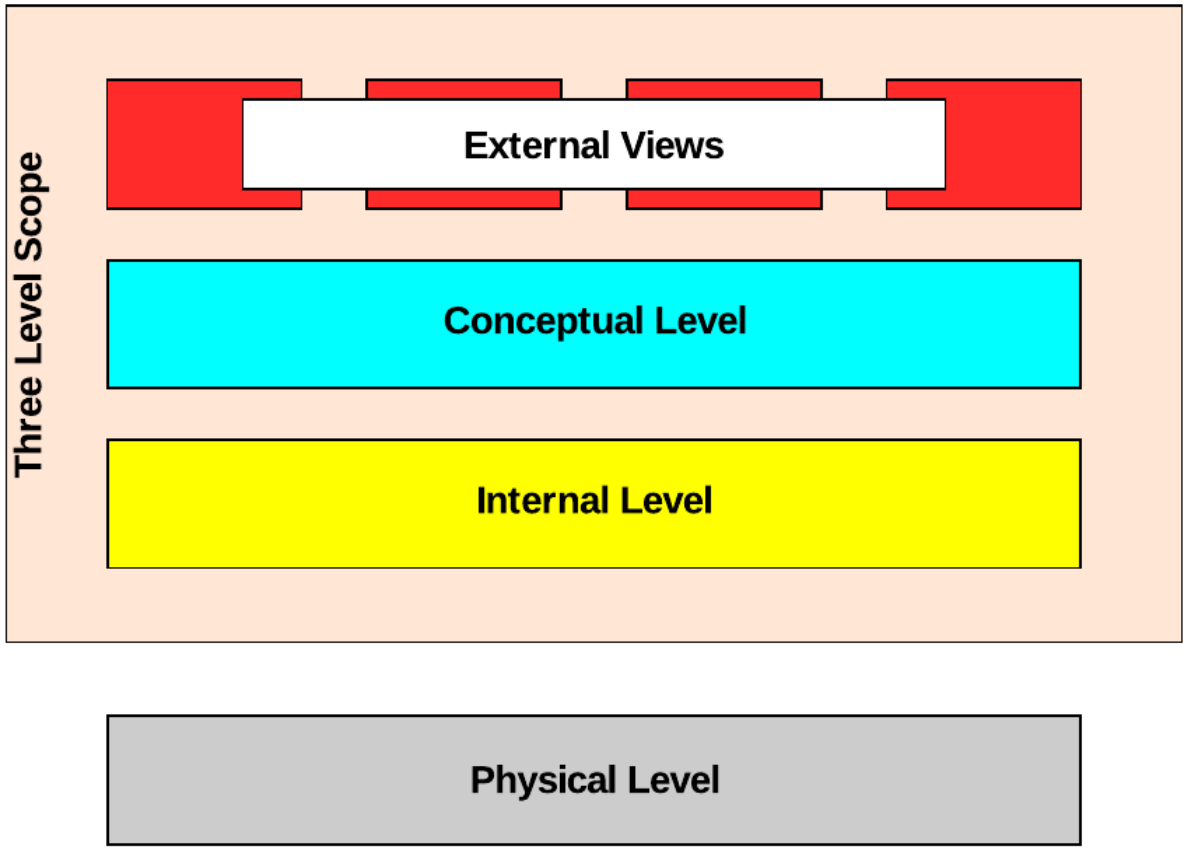
Four Levels of Data

- ANSII-SPARC
  - Formalized into the three level architecture
- External Level
  - Concerned with how the data is viewed by the individual users
    - This implies that each user can have their own unique “model” or how they think the data is organized
    - Corresponds to the domain model
  - The external level is usually generated by some sort of query mechanism
    - They represent a view into the data from a specific perspective



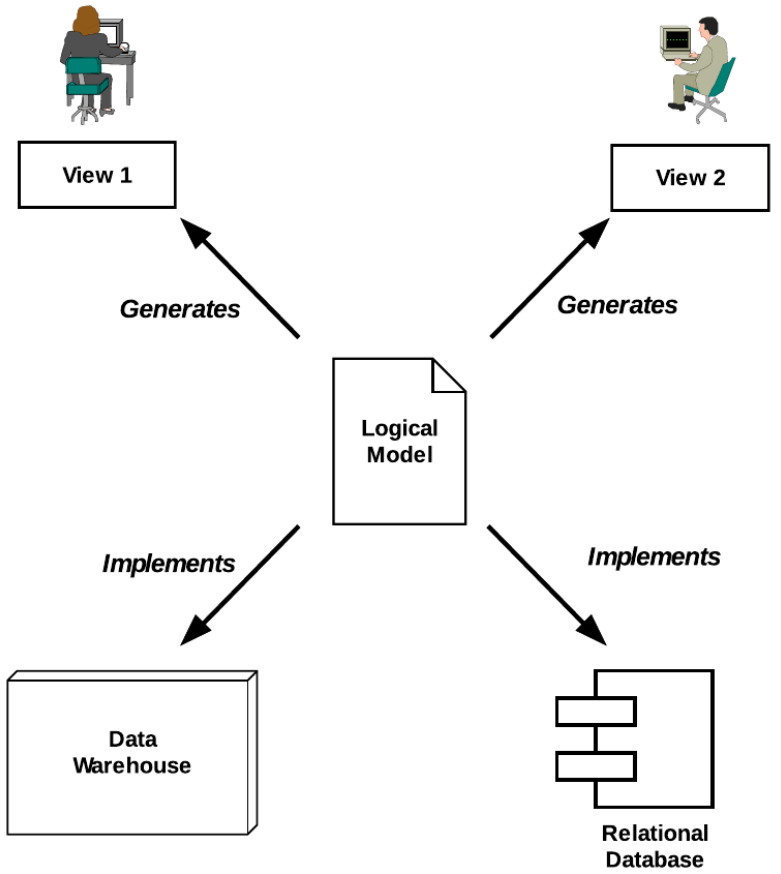
Four Levels of Data

- ANSII-SPARC
- Conceptual level
  - This is the logical view of the data
  - Corresponds to Chen's level 2
  - The single source of truth
  - Expressed in the DSL appropriate for the specific domain the data relates to
  - Does not store data, it just defines the data



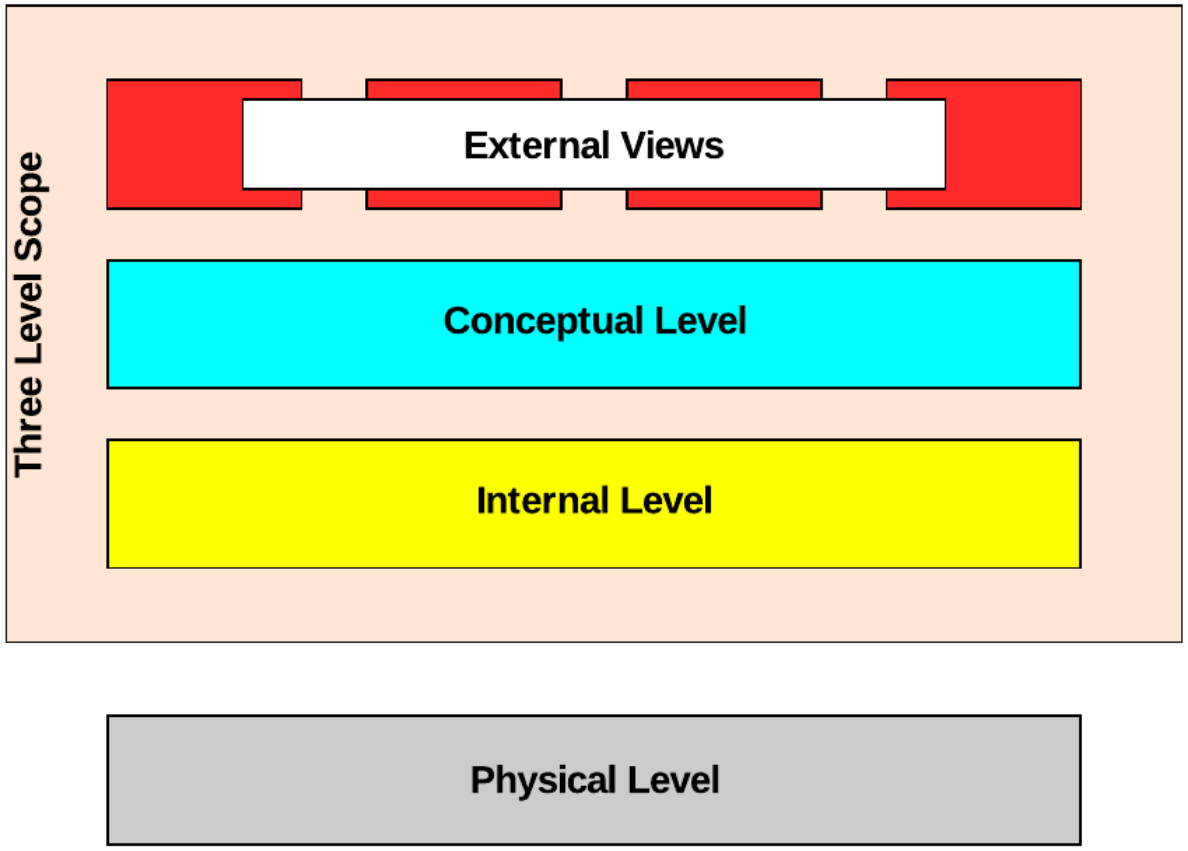
Four Levels of Data

- ANSII-SPARC
- Internal level
  - How the data in the conceptual level is organized for use
  - There may be multiple different implementation models
  - Each model serves a different purpose
  - Data still is not stored here, this level defines the physical shape of the physical data storage



Four Levels of Data

- ANSII-SPARC
- Physical level
  - This is the level that contains the data
  - It is the implementation of shape of the data storage defined at the internal level
  - This is the level where Oracle is differentiated from other vendors like PostgreSQL or IBM DB2
  - Vendors extend the implementation model at the physical level to improve performance and efficiency



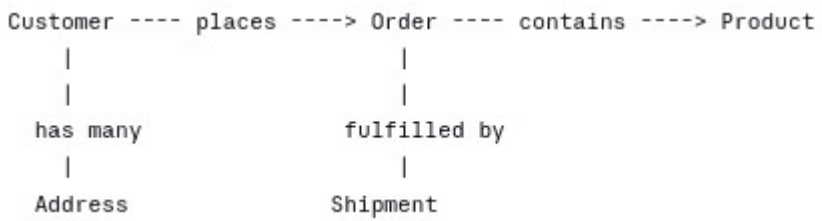
Four Levels of Data

- The domain model
  - Captures the real world
  - Entities are the things that matter
    - The people, places, events, and concepts the domain cares about
    - Relationships describe how those things interact or depend on each other
    - Attributes describe the properties of entities and relationships
  - No tables, no columns, no data types at this stage

Level	Name	Description
1	Information about entities and relationships	Real-world things and their associations as they exist in the mind -- concepts, not data
2	Information structure	How those concepts are organized and represented as data: entities, attributes, relationships, and value sets
3	Access-path-independent data structure	A formal data structure, such as a relational table, that does not specify how data will be physically located or retrieved
4	Access-path-dependent data structure	The physical storage and retrieval structures: indexes, data structure diagrams, storage organization

Four Levels of Data

- Primary activity is discovery
  - Interview domain experts
    - What do they track, what decisions do they make, what events matter?
  - Look for the nouns in domain conversations
    - They often reveal entities
  - Look for the verbs
    - They often reveal relationships
  - Look for the questions the domain needs to answer
    - They reveal what data must be captured
  - Often requires deep dives into the domain to understand the data



Four Levels of Data

- The logical model
  - Creates formal definitions
  - Entities become defined with explicit attributes and data types
  - Relationships are defined with cardinality
    - One-to-one, one-to-many, many-to-many
  - Primary identifiers are established for each entity
  - Business rules are expressed as constraints on the model
  - No technology-specific constructs
    - No Oracle-specific types, no index definitions, no partitioning

Domain Model	Logical Model
Customer places Orders	Customer (customer_id, name, email, status) 1..* Orders
Order contains Products	Order_Line (order_id FK, product_id FK, quantity: integer, unit_price: decimal)
Product has a price	Product (product_id, name, description, list_price: decimal, active: boolean)

Four Levels of Data

- The logical model
  - Can be reviewed and validated without database expertise
  - Can be implemented in Oracle, PostgreSQL, SQL Server, or any relational engine
  - Can also serve as the basis for a non-relational implementation if the use case demands it
  - For example, dimensional models used in data warehouses
- Changes at the logical level are cheap
  - Changes at the physical level are expensive

Domain Model	Logical Model
Customer places Orders	Customer (customer_id, name, email, status) 1..* Orders
Order contains Products	Order_Line (order_id FK, product_id FK, quantity: integer, unit_price: decimal)
Product has a price	Product (product_id, name, description, list_price: decimal, active: boolean)

Four Levels of Data

- The internal model
  - Logical structure becomes a data structure
    - The entities and relationships defined at the logical level are mapped to tables and columns
  - The relational model introduces
    - Tables, primary keys, foreign keys, and constraints
  - "Access-path-independent"
    - Means the structure does not specify how rows will be found
    - No indexes, no storage clauses, no partitioning
  - In Oracle, this corresponds to
    - CREATE TABLE statements with their constraints
    - Before any index or storage decisions are made

Logical Construct	Physical Implementation Options
Many-to-many relationship	Junction table with a composite primary key
Optional attribute	Nullable column or separate child table
Subtype / supertype	Single table with discriminator column, or separate tables with shared PK
Computed attribute	Derived column, virtual column, or materialized view
Large text attribute	VARCHAR2, CLOB, or external storage depending on size

Four Levels of Data

- The physical model
  - Where physical storage and retrieval are defined
  - At this level, the abstract data structures of the implementation model are given concrete physical form
  - Access paths are defined
    - Which indexes exist, how records are stored, how they are linked
  - In Oracle, these physical decisions include
    - Index types and column selection
    - Partitioning strategy, table clustering, storage parameters
    - Parallel query configuration

Level	Name	Description
1	Information about entities and relationships	Real-world things and their associations as they exist in the mind -- concepts, not data
2	Information structure	How those concepts are organized and represented as data: entities, attributes, relationships, and value sets
3	Access-path-independent data structure	A formal data structure, such as a relational table, that does not specify how data will be physically located or retrieved
4	Access-path-dependent data structure	The physical storage and retrieval structures: indexes, data structure diagrams, storage organization

Four Levels of Data

- Schema design
  - Defining tables, columns, constraints and relationships are logical and implementation level activities
  - Index design, partitioning, and storage organization are activities that take place at the physical level
  - Physical level decisions should be driven by measured performance requirements
    - Using a correct logical model and relational schema
    - Not by trying anticipating performance patterns before the schema is correct

Level	Name	Description
1	Information about entities and relationships	Real-world things and their associations as they exist in the mind -- concepts, not data
2	Information structure	How those concepts are organized and represented as data: entities, attributes, relationships, and value sets
3	Access-path-independent data structure	A formal data structure, such as a relational table, that does not specify how data will be physically located or retrieved
4	Access-path-dependent data structure	The physical storage and retrieval structures: indexes, data structure diagrams, storage organization

OLAP and OLTP

- OLTP (Online Transaction Processing)
  - High volume of short, concurrent transactions
  - Operations that read, insert, update, or delete individual or small sets of rows
  - Strong consistency requirements
    - Transactions must be atomic and isolated
  - Data is current
    - Queries operate on the present state of the system
  - Write performance matters as much as read performance

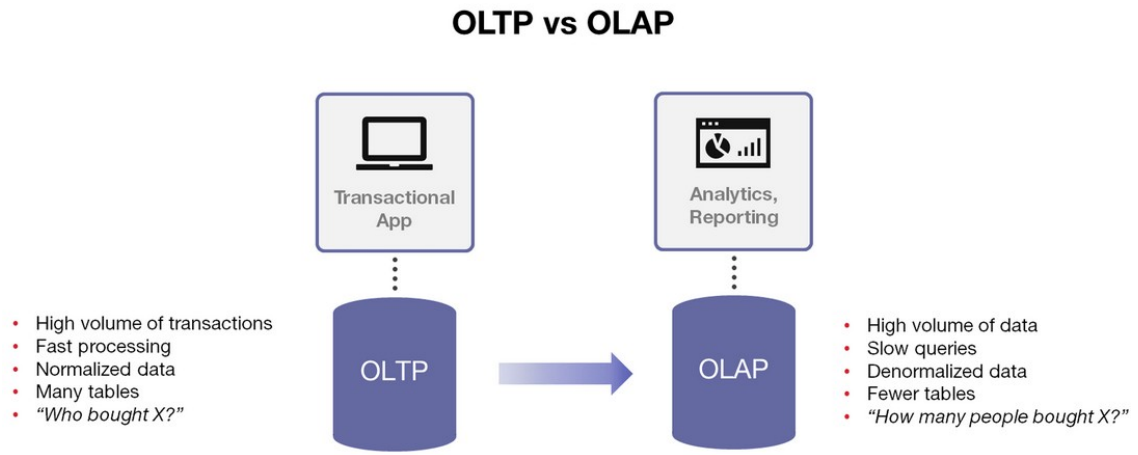


Image Credit: https://hevodata.com/learn/oltp-vs-olap/

OLAP and OLTP

- Normalization
  - Minimizes the data written per transaction so that only the changed fact is updated
  - Row-level locking
    - Allows high concurrency with minimal contention
  - ACID transactions guarantee consistency even under concurrent access
  - Indexes provide fast, selective access to individual rows or small ranges
  - Foreign key constraints enforce referential integrity within the transactions

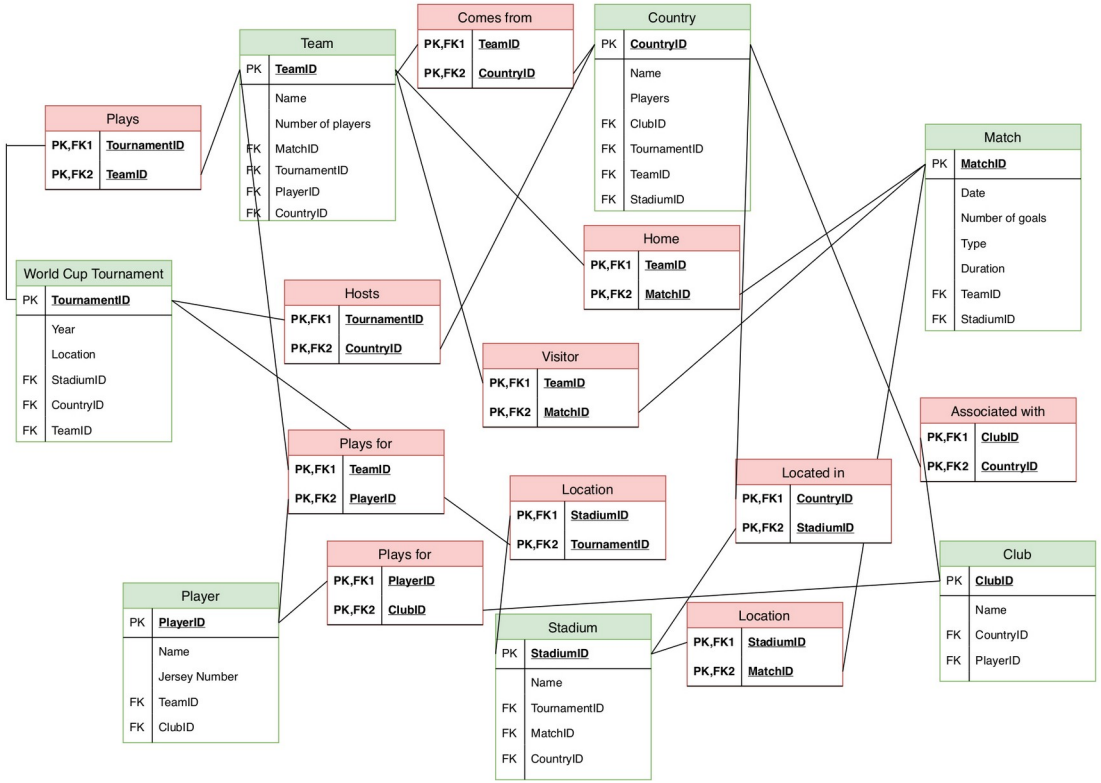


Image Credit: https://storage.googleapis.com/dajhwfuteugmme/what-is-data-entity-model.html

OLAP and OLTP

- Normalization
  - Many narrow, normalized tables
  - Surrogate integer primary keys
  - Foreign keys enforcing relationships
  - Indexes on lookup and join columns
  - Relatively shallow queries
    - Few joins, small result sets
  - No pre-aggregated data
    - Totals, averages and other computations are always computed from current facts
    - Avoids stale and misleading computations

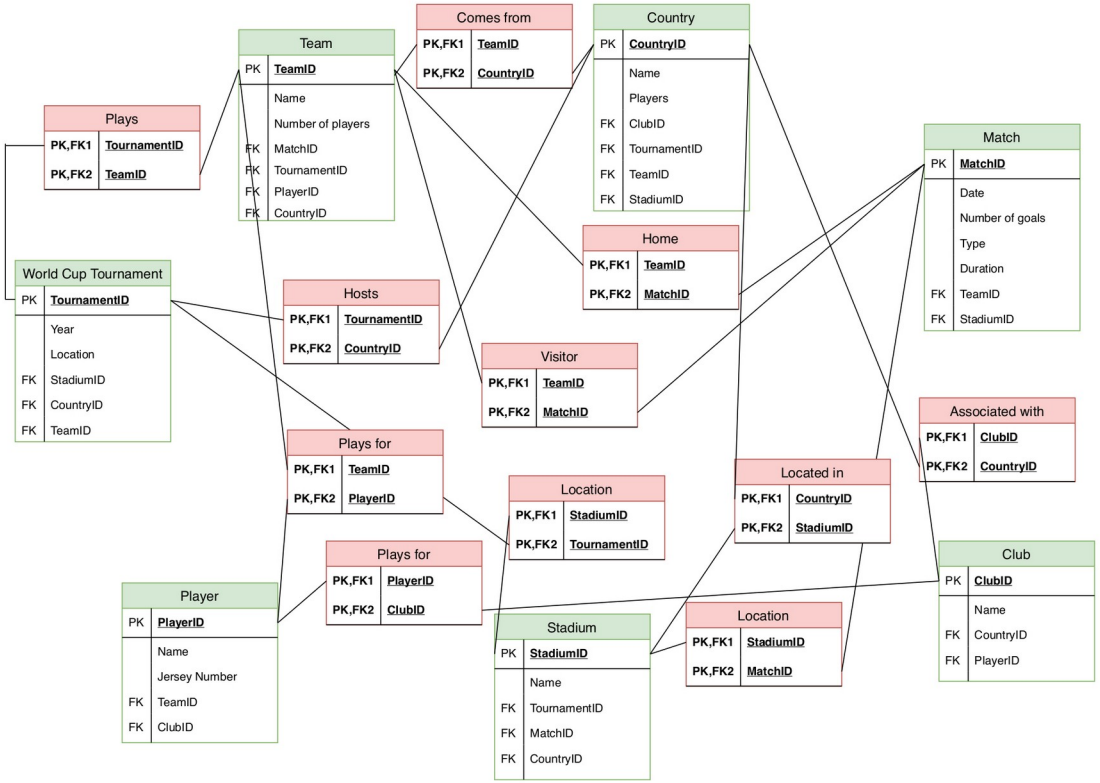


Image Credit: https://storage.googleapis.com/dajhwfuteugmme/what-is-data-entity-model.html

OLAP and OLTP

- OLAP (Online Analytical Processing)
  - Low volume of complex, long-running queries
  - Operations that aggregate, group, and summarize large volumes of data
  - Queries span historical data
    - "What happened over the last three years?"
  - Read-heavy
    - Data is loaded in bulk, not updated row by row
  - Write performance concerns are negligible compared to query performance on large aggregations

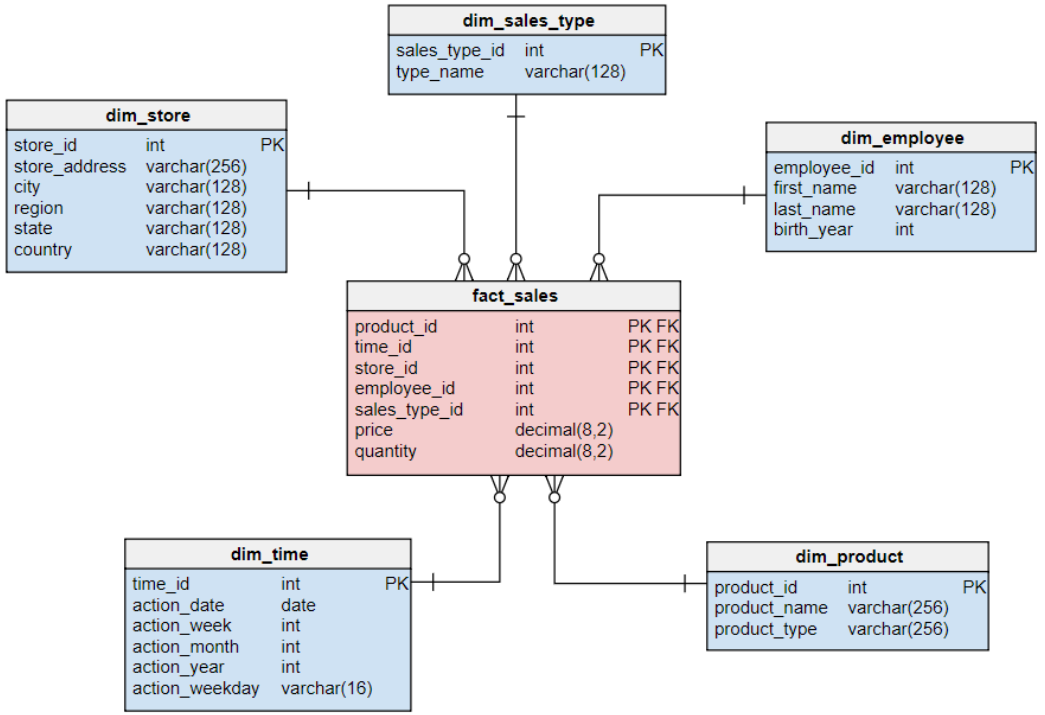


Image Credit: https://xaydungso.vn/blog5/dimensional-modeling-data-warehouse-vi-cb.html

OLAP and OLTP

- Why relational models struggle
  - Normalized schema require many joins to reconstruct data aggregates
  - Joins across billions of rows are expensive even with good indexes
  - Query patterns are ad-hoc and unpredictable
    - Indexing strategies that work for OLTP do not generalize
  - OLTP schema optimize for write efficiency
  - OLAP shema optimize for read aggregation

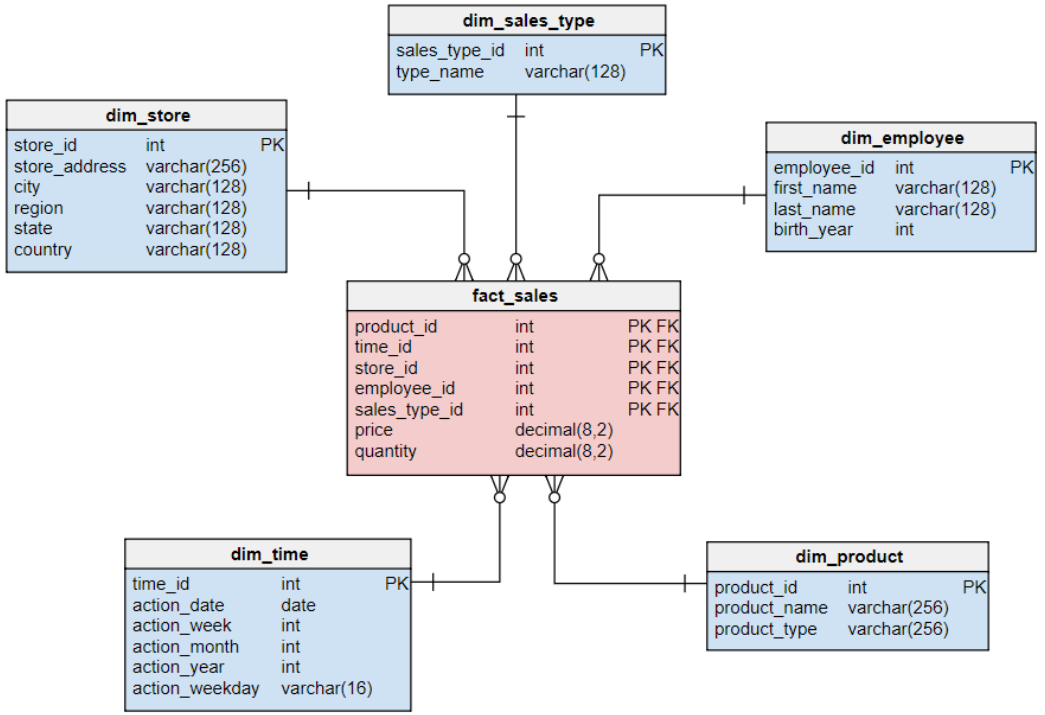


Image Credit: https://xaydungso.vn/blog5/dimensional-modeling-data-warehouse-vi-cb.html

OLAP and OLTP

- The dimensional OLAP model (Kimball)
  - Fact tables
    - Store measurable events at the grain of the business process like a sale, transaction, or call
  - Dimension tables
    - Store the descriptive context of those events
    - Who, what, where, when, why
  - Star schema
    - One central fact table surrounded by dimension tables
    - Minimal joins, fast aggregation

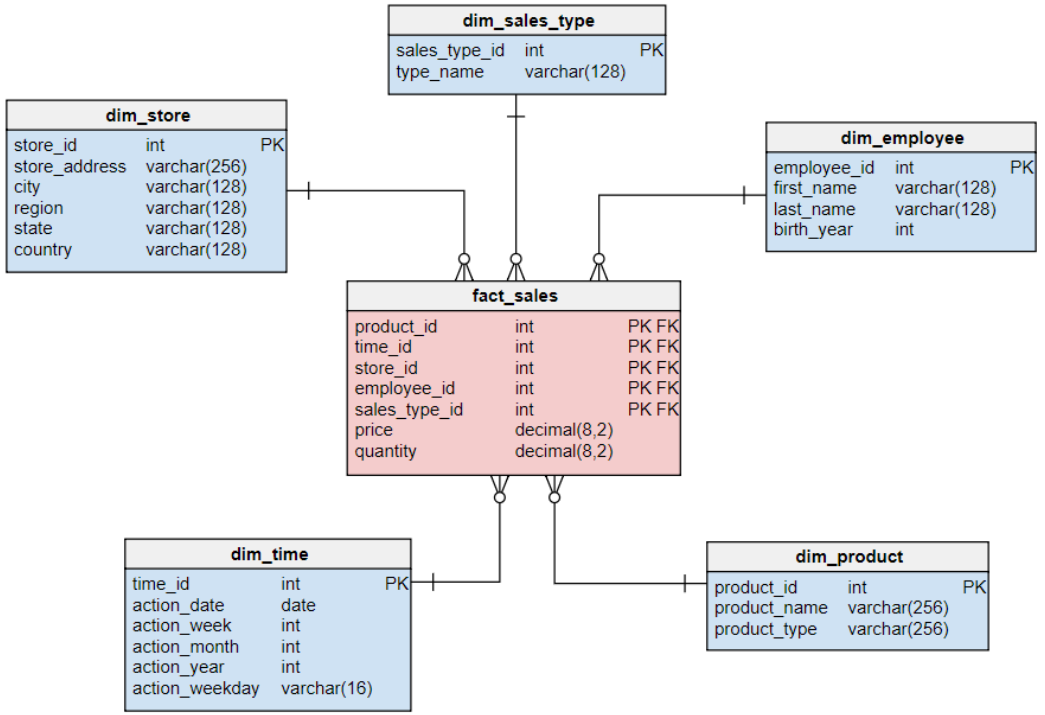


Image Credit: https://xaydungso.vn/blog5/dimensional-modeling-data-warehouse-vi-cb.html

OLAP and OLTP

- Fact table characteristics
  - One row per measurable event
  - Contains foreign keys to all relevant dimension tables
  - Contains the numeric measure
    - For example: amount, quantity, duration, count
  - Very wide (many columns) and very tall (billions of rows in large systems)
  - Append-only
    - Historical facts are not updated

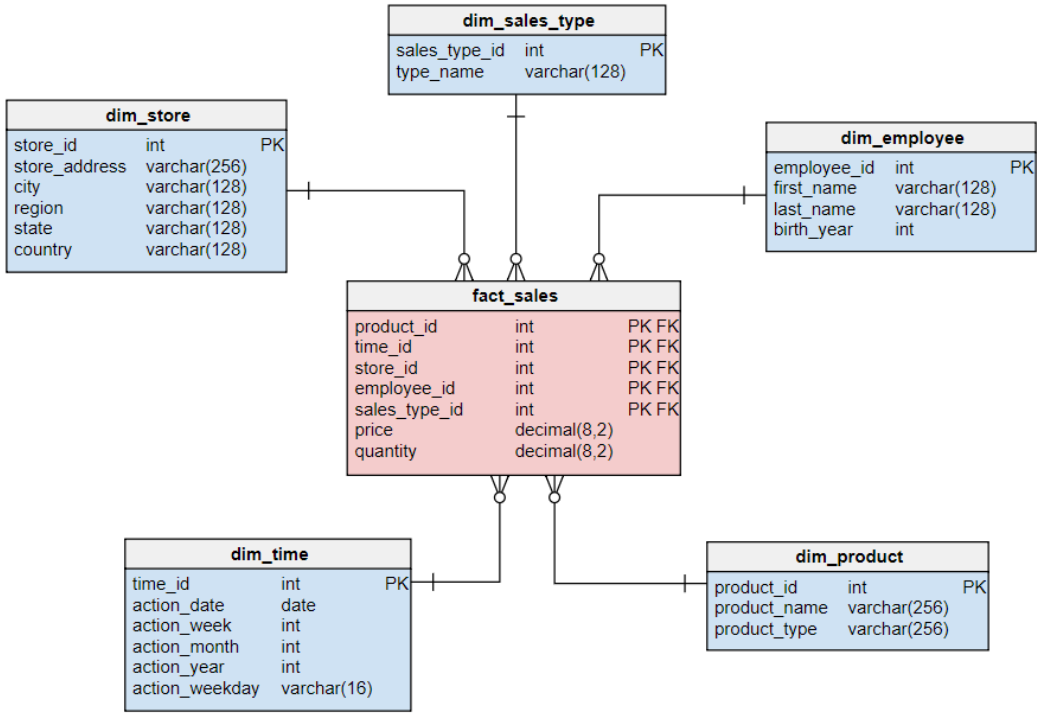


Image Credit: https://xaydungso.vn/blog5/dimensional-modeling-data-warehouse-vi-cb.html

OLAP and OLTP

- Dimension table characteristics
  - Contain descriptive attributes
    - Names, categories, hierarchies, labels
  - Relatively small
    - Thousands to millions of rows
  - Denormalized intentionally
    - All attributes for a dimension are in one table
  - Support filtering and grouping in queries

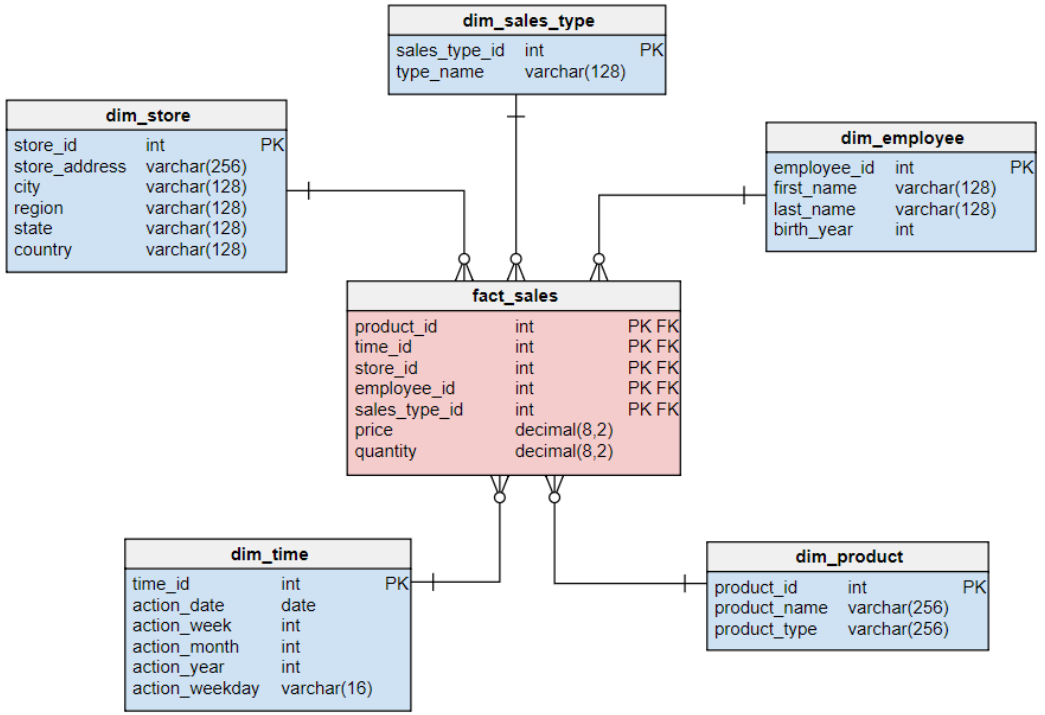


Image Credit: https://xaydungso.vn/blog5/dimensional-modeling-data-warehouse-vi-cb.html

OLAP and OLTP

- Denormalization of dimension tables
  - In the relational model
    - A product has a category, a category has a department
    - Requires three tables and two joins
  - In a dimension table
    - Product_name, category_name, and department_name are all columns in DIM_PRODUCT
  - This collapses joins and makes queries dramatically simpler and faster
  - The trade-off
    - Update anomalies can occur
    - Given the data is historical, the only expected updates are to perform error corrections in the data

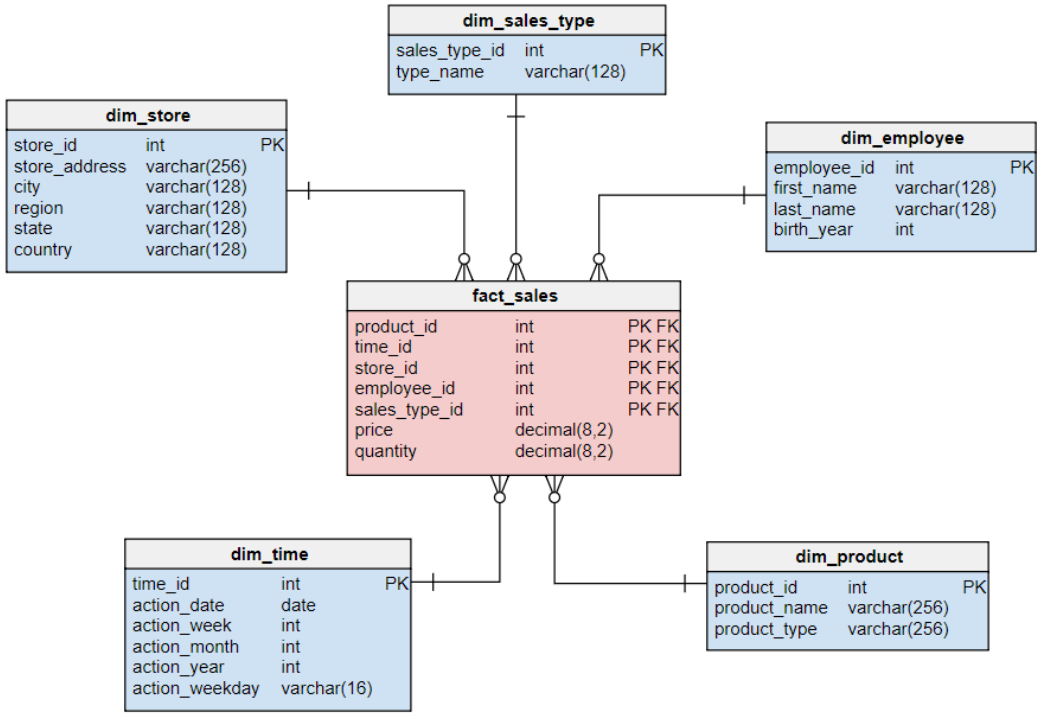


Image Credit: https://xaydungso.vn/blog5/dimensional-modeling-data-warehouse-vi-cb.html

OLAP and OLTP

- Common enterprise architectural pattern
  - A normalized OLTP schema serves operational applications
  - Data is extracted, transformed, and loaded (ETL) into a dimensional model for analytics
  - The two schemas are maintained separately and optimized independently
  - Oracle supports both patterns and provides tools for each
- The mistake to avoid
  - Analytical queries run directly against a normalized OLTP schema perform poorly
  - OLTP schemas modified with denormalization to serve analytics degrade transactional performance
  - Treat OLTP and OLAP as separate concerns with separate implementations

Dimension	Relational (OLTP)	Dimensional (OLAP)
Primary use	Transactional operations	Analytical queries
Schema style	Normalized (3NF)	Denormalized (star or snowflake)
Query pattern	Selective, point-in-time	Aggregate, historical range
Write pattern	Frequent, small, concurrent	Bulk loads, periodic
Join depth	Many joins, small result sets	Few joins (star), large scans
Consistency requirement	Strong (ACID)	Eventual or point-in-time
Index strategy	Selective B-tree indexes	Bitmap indexes, partition pruning
Oracle features used	Constraints, row locking, sequences	Partitioning, parallel query, materialized views

Oracle Tables and Storage

Relational Tables

- The formal definition
  - A table (relation) is a set of tuples, all conforming to the same schema
  - Each column represents one attribute of the entity or relationship being modeled
  - Each row represents one instance of that entity or relationship
  - No two rows may be identical
    - The primary key enforces this
  - Row order is not defined
    - A table is a set, not a list

Spreadsheet	Relational Table
Row order is meaningful	Row order is undefined
Column order is meaningful	Column order is irrelevant to queries
No enforcement of uniqueness	Primary key enforces uniqueness
No enforcement of data types	Column data types are enforced
Computed values mixed with raw data	Derived values belong in views or queries
No referential integrity	Foreign keys enforce relationships

Relational Tables

- Oracle storage
  - Oracle stores table data in data blocks (default 8KB each)
  - Blocks are grouped into extents, and extents into segments
  - A heap-organized table stores rows in no particular order
    - Rows land wherever space is available
  - Row order in query results is never guaranteed without an explicit ORDER BY clause
    - Consequence of the set-theoretic foundation

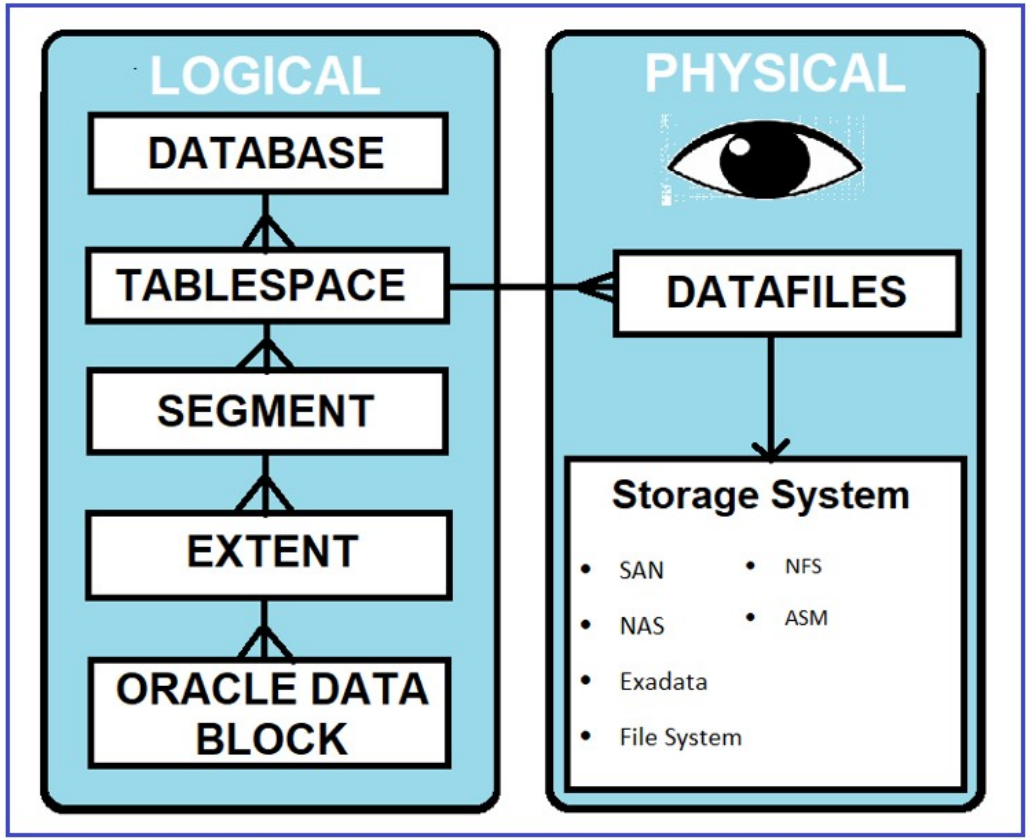


Image Credit:https://dotnettutorials.net/lesson/oracle-storage-structures/

Relational Tables

- How rows are stored
  - Oracle reads and writes data in blocks, not individual rows
  - Default block size is 8KB
    - Enterprise configurations may use 16KB or 32KB blocks
  - A single block may contain many rows
    - A wide row may span multiple blocks
    - Called row chaining
  - The optimizer's cost estimates are partly based on how many blocks must be read

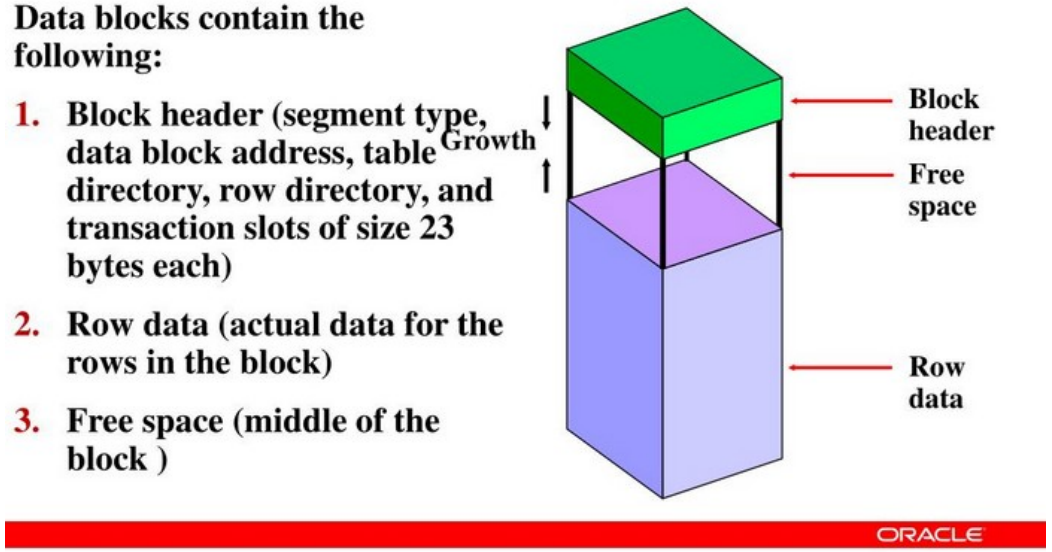


Image Credit: https://www.slideserve.com/jennis/lecture-4-oracle-tablespaces

Relational Tables

- Updates
  - If an UPDATE causes a row to grow beyond the free space remaining in its block
  - Oracle moves it to a new block and leaves a forwarding pointer
    - This is called row migration
  - Require extra I/O to retrieve a single row
  - Should be avoided through good column sizing and appropriate PCTFREE settings

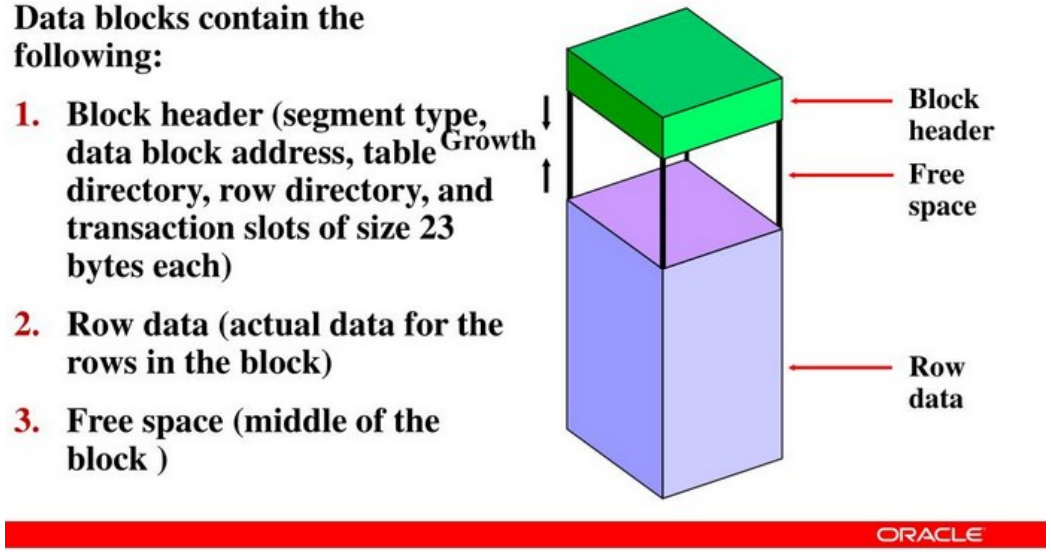


Image Credit: https://www.slideserve.com/jennis/lecture-4-oracle-tablespaces

Relational Tables

- Heap tables
  - Are the default
  - Rows are inserted into any available free space in the segment
  - Deleted rows leave gaps
    - Oracle reuses that space for future inserts
  - There is no physical clustering of related rows unless explicitly configured
  - This is the correct mode for almost all use cases

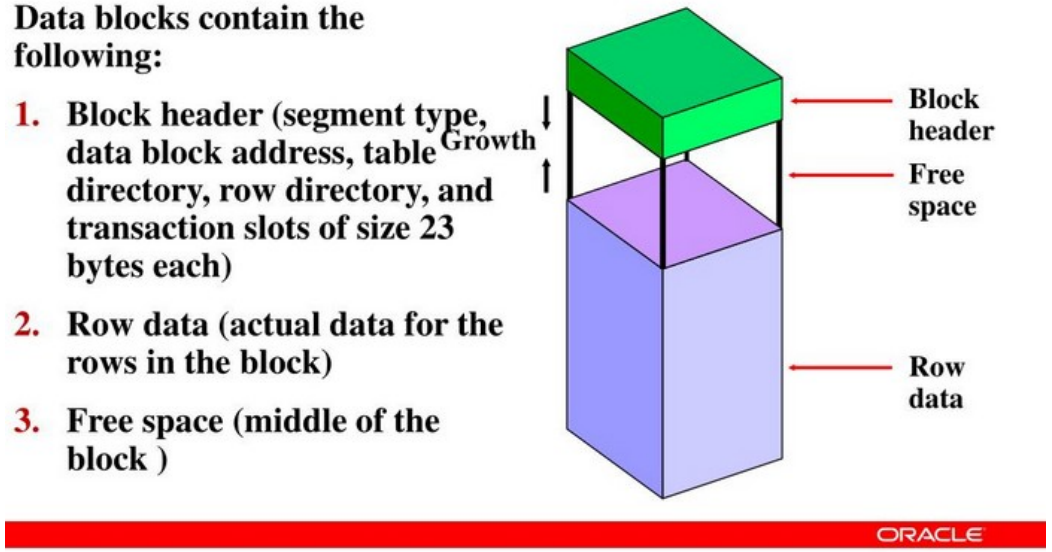


Image Credit: https://www.slideserve.com/jennis/lecture-4-oracle-tablespaces

Relational Tables

- Heap tables
  - A heap is an unordered collection
  - Rows are placed wherever free space is available in the segment
  - No relationship between the order rows were inserted and the order they are physically stored
  - After deletions and updates, rows from different logical groupings are interleaved in the same blocks
  - Correct structure for workloads that insert, update, and delete concurrently

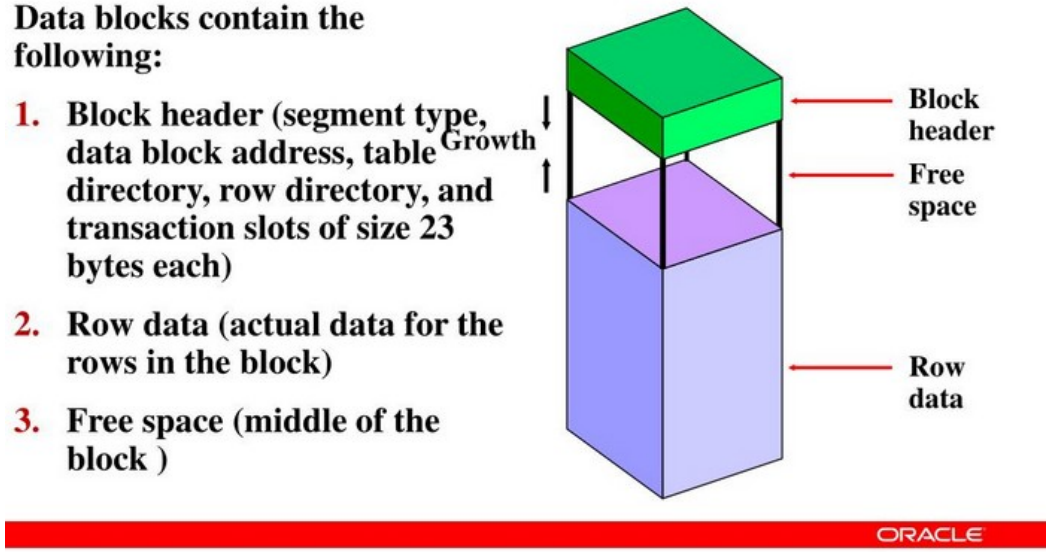


Image Credit: https://www.slideserve.com/jennis/lecture-4-oracle-tablespaces

Relational Tables

- How inserts work in a heap table
  - Oracle maintains a freelist to track blocks with available space
    - Modern configuration use Automatic Segment Space Management, ASSM, in modern configurations
    - Tracks any block with sufficient free space
  - Under high concurrent insert load, multiple sessions write to different blocks simultaneously with minimal contention
  - There is no serialization point that would force all inserts to go to the same location
  - Supports parallelism in operations
  - Improves overall CRUD performance

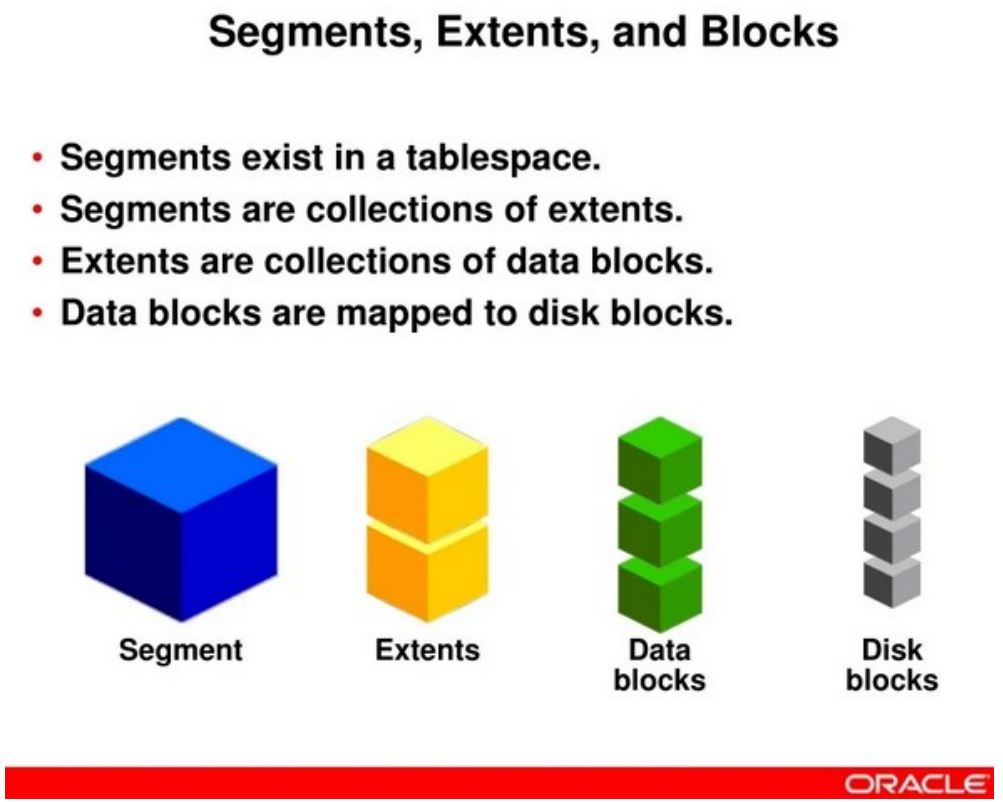


Image Credit: https://www.slideserve.com/jennis/lecture-4-oracle-tablespaces

Relational Tables

- How Oracle finds rows in a heap table
  - Without an index
  - Oracle reads every block in the segment for a *full table scan*
  - Efficient for large result sets when a large percentage of the rows must be returned
  - Seems counter intuitive but at larger percentages of rows being fetched
  - The cost of per row lookups from an index exceeds the cost of scanning all blocks

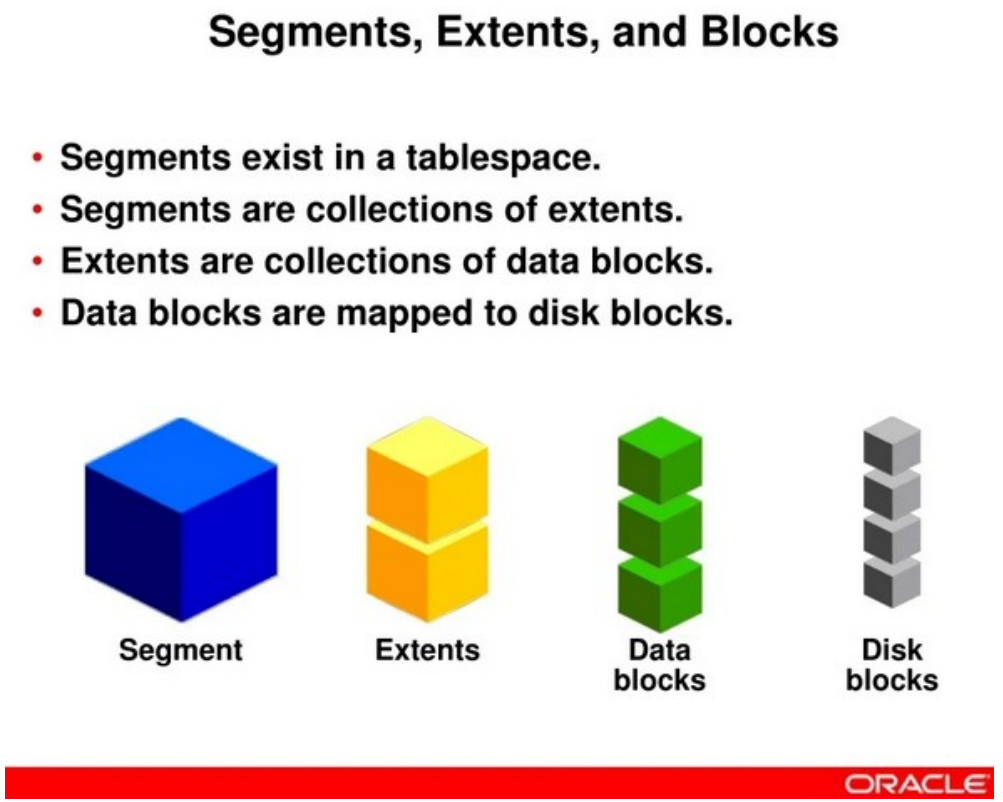


Image Credit: https://www.slideserve.com/jennis/lecture-4-oracle-tablespaces

Relational Tables

- How Oracle finds rows in a heap table
  - With an index
    - Oracle finds the ROWID in the index and uses it to go directly to the specific block
    - Efficient for small, selective result sets
  - The optimizer chooses between these two access path
    - This is based on the estimated number of rows to be returned and the cost of each approach

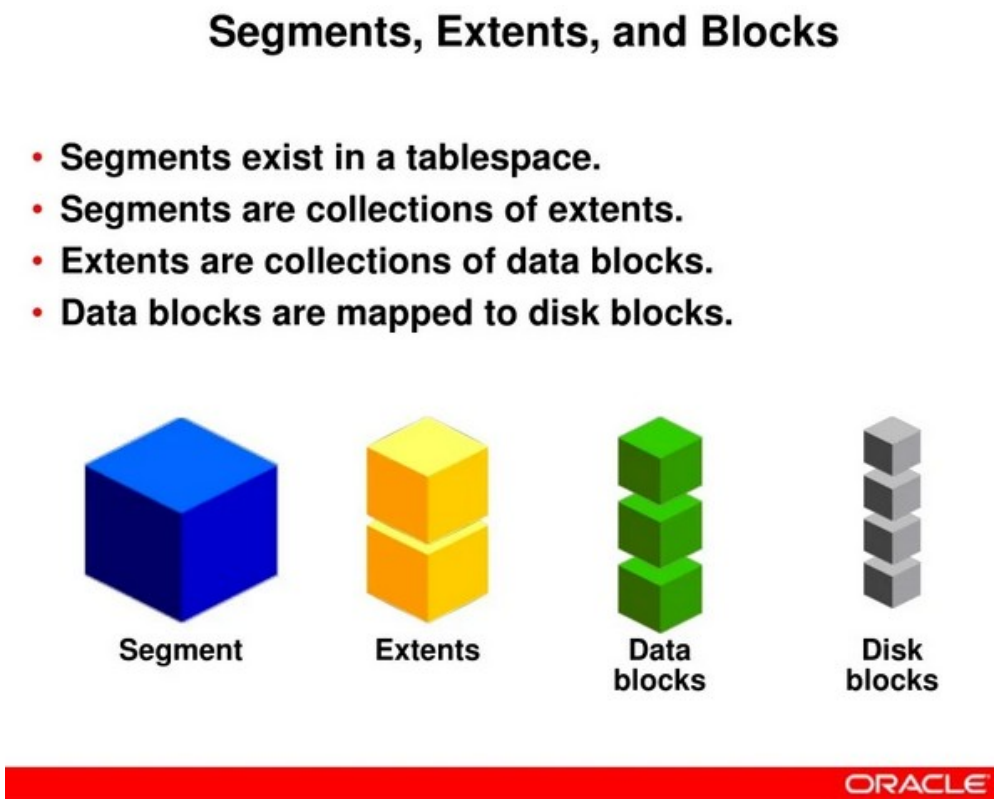


Image Credit: https://www.slideserve.com/jennis/lecture-4-oracle-tablespaces

Relational Tables

- Oracle query optimizer
  - SQL is a declarative language
    - A query says what data to retrieve, not how to retrieve it
  - For any given SQL statement
    - Multiple valid ways to physically execute it
    - For example, different join orders, different access paths, different algorithms
  - The optimizer evaluates those alternatives and chooses the one with the lowest estimated cost

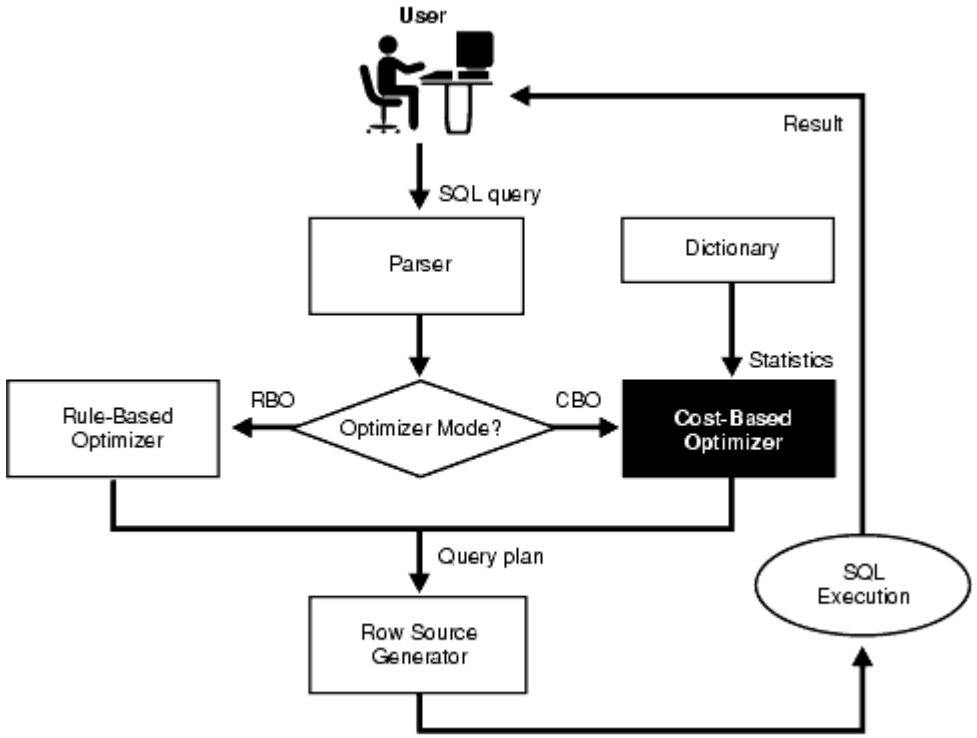


Image Credit: https://docs.oracle.com/cd/B10500_01/server.920/a96533/optimops.htm

Relational Tables

- Oracle query optimizer
  - Runs automatically before every query executes
  - It receives a parsed SQL statement and produces an execution plan
    - A specific, ordered sequence of physical operations that will produce the correct result
  - It is cost-based
    - Every candidate plan is assigned a numeric cost derived from statistics about the data, and the plan with the lowest cost is selected
  - It runs in microseconds
    - The developer never waits for optimization to happen

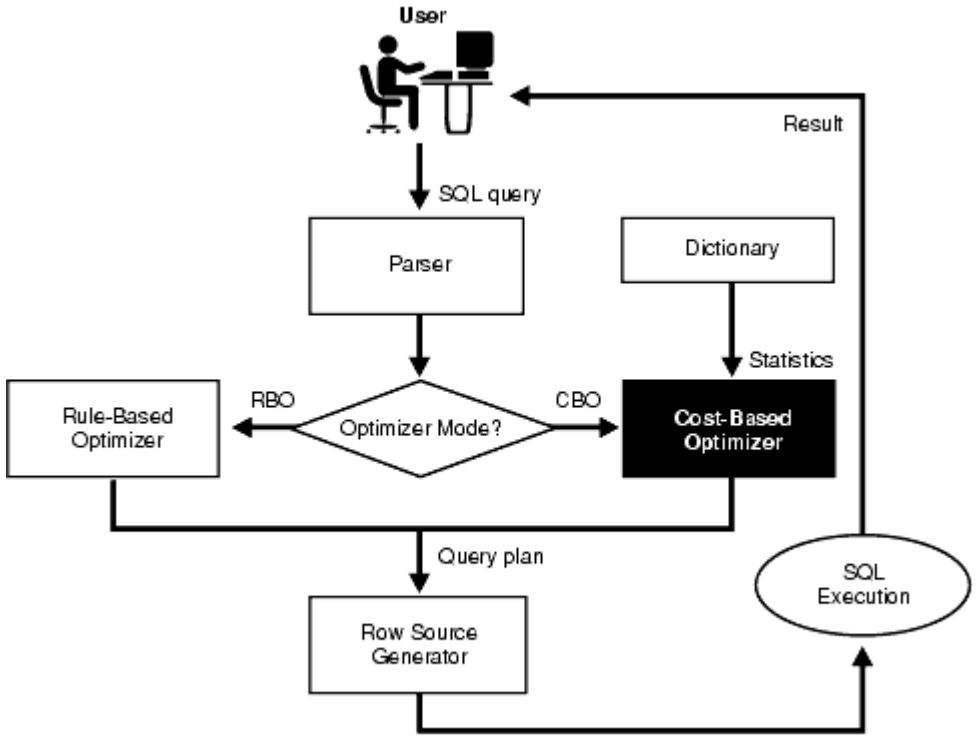


Image Credit: https://docs.oracle.com/cd/B10500_01/server.920/a96533/optimops.htm

Relational Tables

- What "cost" means
  - Oracle's internal estimate of the total work required to execute a plan
  - Expressed in units of I/O and CPU consumption, weighted together
    - A plan that reads 200 blocks has a lower cost than one that reads 2,000 blocks, all else being equal
  - Cost is an estimate, not a guarantee
    - It is only as accurate as the statistics the optimizer has access to
    - Stale statistics produce inaccurate cost estimations

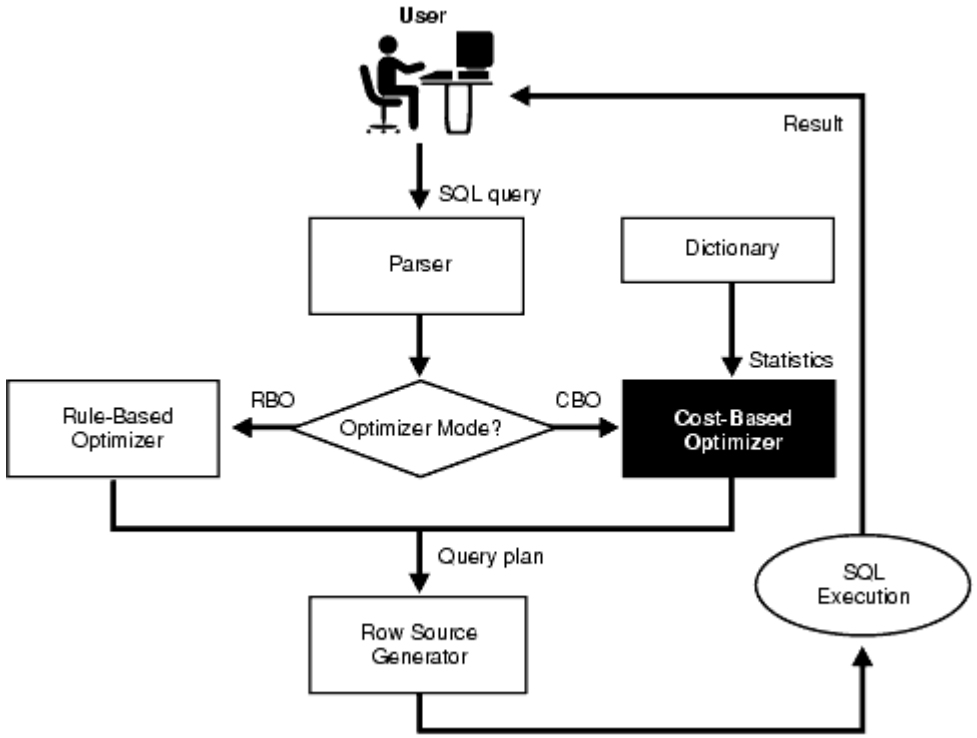


Image Credit: https://docs.oracle.com/cd/B10500_01/server.920/a96533/optimops.htm

Relational Tables

- The optimizer's inputs
  - The SQL statement itself
    - Predicates, joins, and requested columns
  - Table and index statistics
    - Row counts, column value distributions, index depth, clustering factors
  - System statistics
    - The relative speed of CPU versus I/O on the current hardware
  - Constraints
    - Primary keys and NOT NULL constraints tell the optimizer facts about the data it can exploit

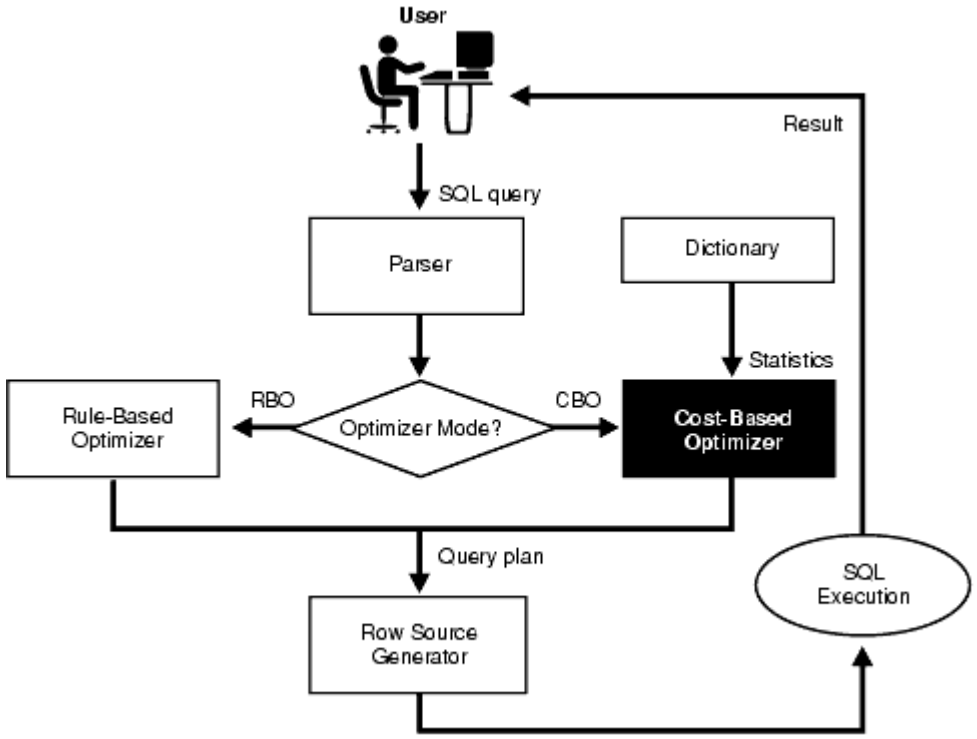


Image Credit: https://docs.oracle.com/cd/B10500_01/server.920/a96533/optimops.htm

Relational Tables

- Statistics critical dependency
  - Without accurate statistics, the optimizer's cost estimates are wrong
  - Wrong estimates lead to wrong plan choices
    - For example, a plan chosen for 1,000 rows that actually returns 10 million rows will perform catastrophically
  - Statistics are gathered automatically by Oracle on a schedule, and manually via DBMS_STATS after significant data changes

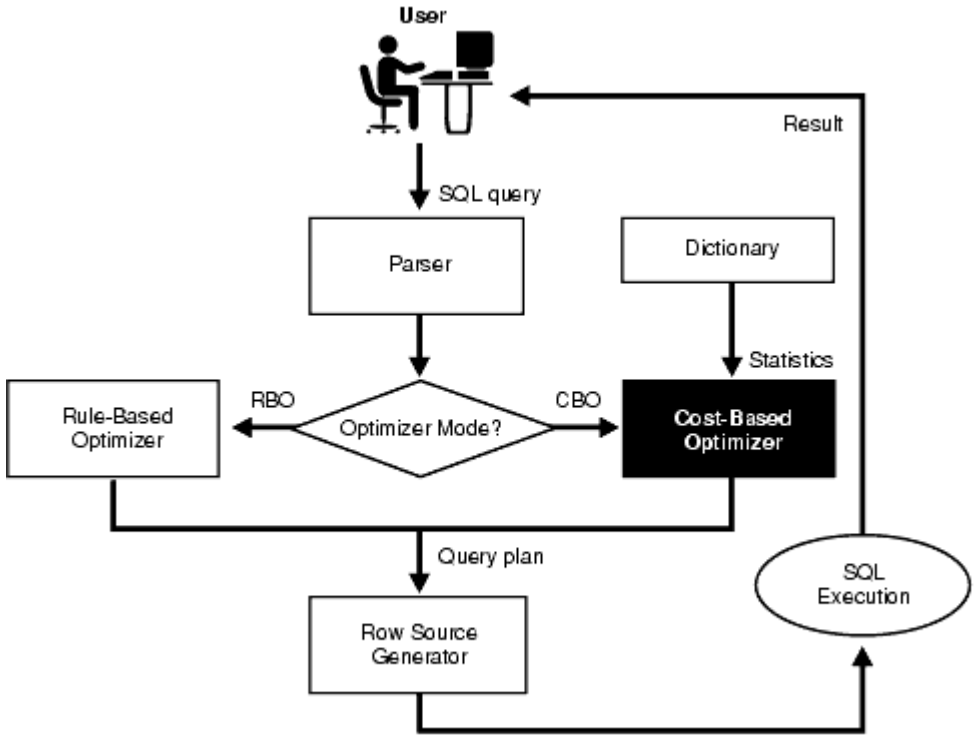


Image Credit: https://docs.oracle.com/cd/B10500_01/server.920/a96533/optimops.htm

Relational Tables

- Advantages of heap tables
  - Write performance
    - Inserts distribute across many blocks, maximizing concurrent throughput
  - Flexibility
    - Any query pattern can be supported by adding appropriate indexes
  - Simplicity
    - No constraints on key values, key updates, or row size relative to block size
  - Compatibility:
    - All Oracle features work fully with heap tables

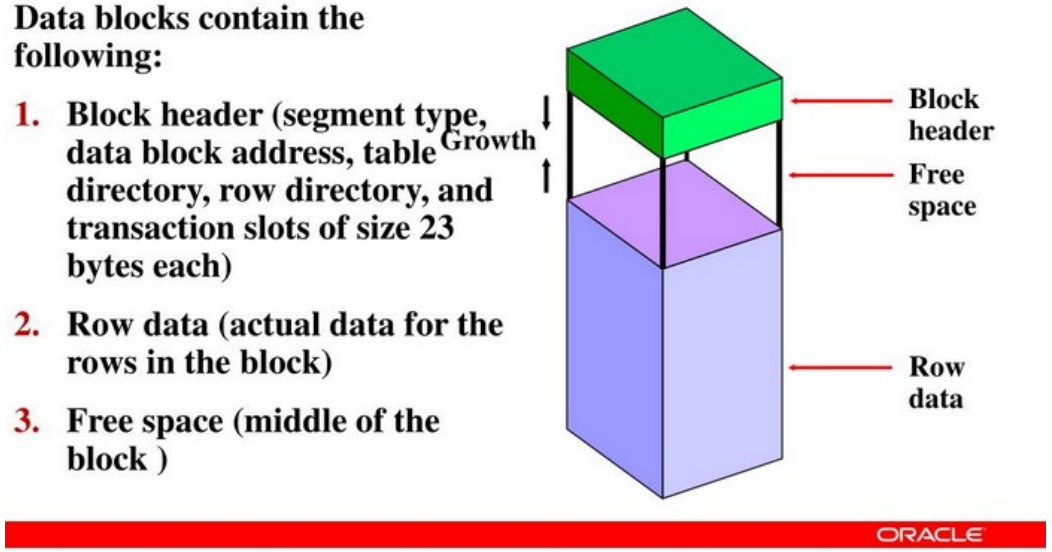


Image Credit: https://www.slideserve.com/jennis/lecture-4-oracle-tablespaces

Relational Tables

- Index-organized tables (IOT)
  - Alternative storage scheme
  - Rows are physically stored in primary key order within a B-tree structure
  - Eliminates the need to store the table data separately from the primary key index
  - Appropriate for tables that are almost always accessed by primary key
  - Poor choice for tables with many secondary access patterns

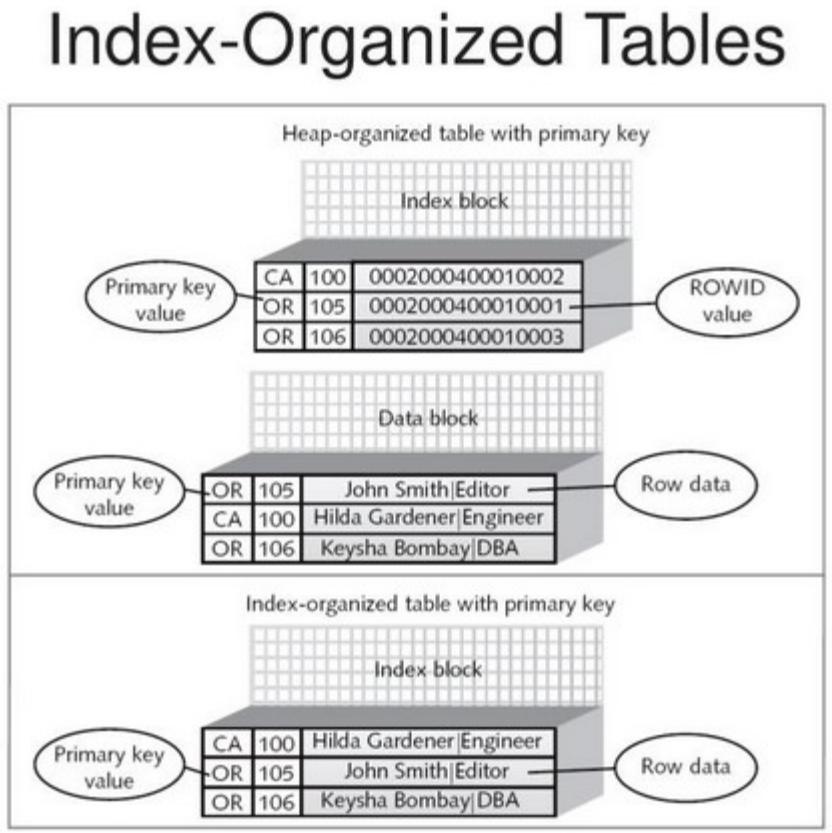


Image Credit: https://www.slideserve.com/hedley-hewitt/oracle-10g-database-administrator-implementation-and-administration

Relational Tables

- The structural difference
  - In a heap table
    - The primary key index and the table data are two separate structures
    - The index stores key values and ROWIDs
    - Oracle does two lookups, one into the index, one into the table
  - In an IOT
    - There is no separate heap
    - The entire row is stored inside the B-tree leaf node, ordered by primary key
    - There is only one structure and one lookup

Index-Organized Tables

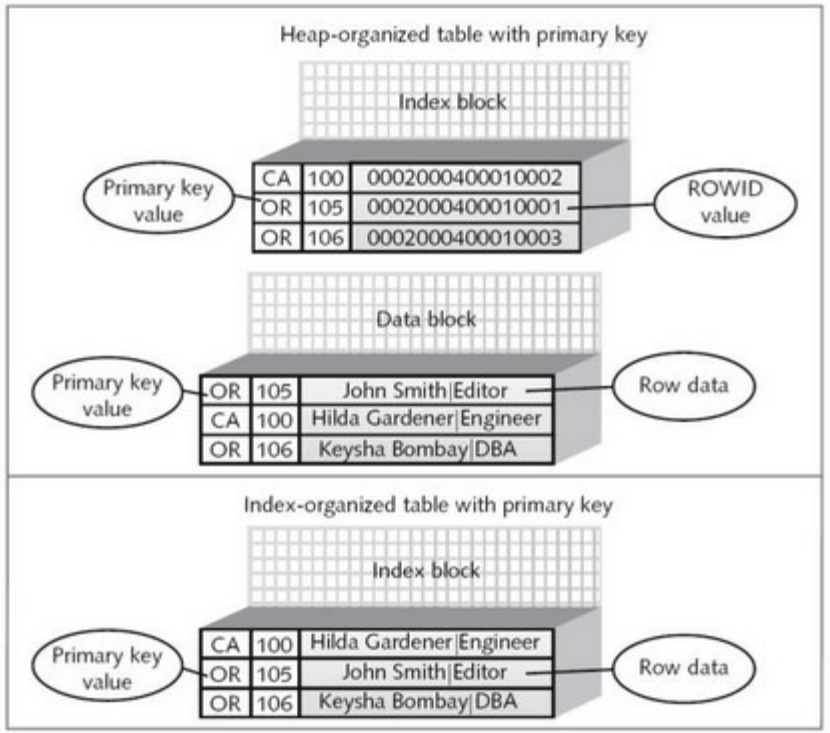


Image Credit: https://www.slideserve.com/hedley-hewitt/oracle-10g-database-administrator-implementation-and-administration

Relational Tables

- Heap table
  - Secondary indexes store the ROWID of the indexed row which is a stable physical pointer
- IOT
  - Secondary indexes cannot store ROWID
    - Rows move when the B-tree rebalances
  - Secondary indexes store the primary key value of the indexed row as the logical pointer
  - To retrieve a row via a secondary index
    - Find the primary key in the secondary index
    - Traverse the IOT B-tree to find the row
    - Two B-tree traversals instead of one index lookup plus one table block read
  - This makes secondary index access on IOTs more expensive than on heap tables

```
Heap Table access by primary key:
  B-tree index leaf: [key=1001, ROWID=AAA.003.0012]
       |
       v  (second I/O: go to the table block)
  Table block: [1001 | Smith | Active | 2024-01-15 | ...]

IOT access by primary key:
  B-tree leaf node: [1001 | Smith | Active | 2024-01-15 | ...]
       |
       (no second I/O -- the row is already here)
```

```
CREATE TABLE country_codes (
    country_code CHAR(2)      NOT NULL,
    country_name VARCHAR2(100) NOT NULL,
    region       VARCHAR2(50),
    CONSTRAINT pk_country_codes PRIMARY KEY (country_code)
) ORGANIZATION INDEX;
```

Relational Tables

- IOT are the right choice when
  - The table is accessed almost exclusively by primary key equality or primary key range scan
  - Secondary access patterns are rare or nonexistent
  - The table is read-heavy with infrequent inserts
    - Primary key values are monotonically increasing which helps avoids B-tree contention
  - The table is small and functions as a lookup table where the primary key carries meaning
  - Storage reduction matters

```
Heap Table access by primary key:
  B-tree index leaf: [key=1001, ROWID=AAA.003.0012]
       |
       v  (second I/O: go to the table block)
  Table block: [1001 | Smith | Active | 2024-01-15 | ...]

IOT access by primary key:
  B-tree leaf node: [1001 | Smith | Active | 2024-01-15 | ...]
       |
       (no second I/O -- the row is already here)
```

```
CREATE TABLE country_codes (
    country_code CHAR(2)      NOT NULL,
    country_name VARCHAR2(100) NOT NULL,
    region       VARCHAR2(50),
    CONSTRAINT pk_country_codes PRIMARY KEY (country_code)
) ORGANIZATION INDEX;
```

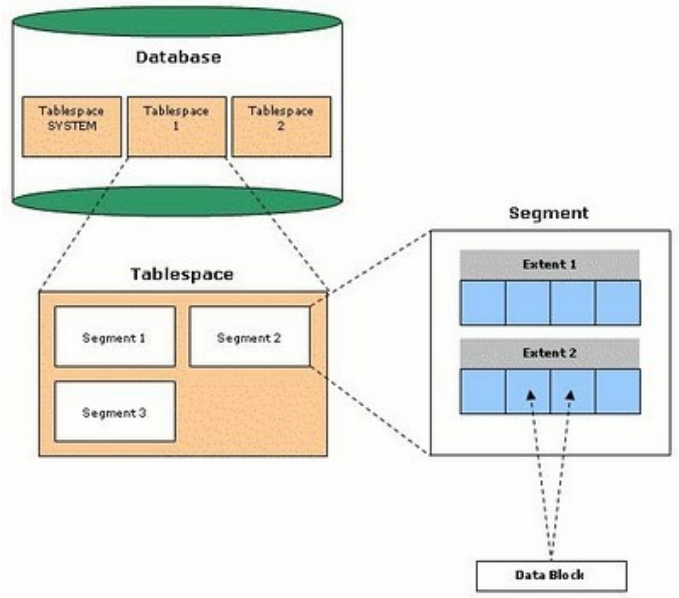
Relational Tables

- When heap tables are the right choice
- Multiple access patterns exist
  - Queries filter on different columns
  - The table receives high concurrent insert volume
  - Secondary indexes will be needed now or in the future
  - The table will be partitioned or may require online redefinition
  - The primary key is a surrogate integer with no range scan value

Characteristic	Heap Table	Index-Organized Table
Physical row order	Unordered (arrival order)	Always sorted by primary key
Primary key lookup	Two lookups: index then table	One lookup: row is in the B-tree leaf
Secondary index access	Fast: index stores stable ROWID	Slower: index stores primary key; two B-tree traversals required
Insert performance	Excellent: distributes across free blocks	Good, but B-tree splits under heavy insert load can cause contention
Update performance	Excellent for most updates	Poor if updates change the primary key (requires delete and re-insert in B-tree)
Range scan by primary key	Requires index range scan plus scattered block reads (high clustering factor cost)	Excellent: rows are physically contiguous in B-tree order
Space efficiency	Stores data and index separately: some overhead	Eliminates separate table segment; saves storage for narrow tables
Concurrent access	Excellent: ASSM distributes inserts	Can suffer B-tree root block contention under high concurrent insert load
Partitioning support	Full support	Limited: only range and list partitioning
Online redefinition	Full support	Not supported

Oracle Storage Hierarchy

- Oracle organizes physical storage in three nested containers
- Tablespace
  - The highest-level logical container for database storage
  - A tablespace is a named group of one or more physical data files on disk
  - Every table and index is assigned to a tablespace at creation time
  - Tablespaces used to separate different classes of data
    - Application data, indexes, temporary sort space, undo data
  - Tablespace is rarely specified explicitly
    - The default is usually correct




```
Tablespace  (logical container, maps to one or more data files on
disk)
  └── Segment  (one per table, one per index)
        └── Extent  (one or more contiguous groups of blocks)
              └── Block  (the unit of I/O -- 8KB by default)
```

Oracle Storage Hierarchy

- Tablespace
  - Decouples logical database objects from the physical files
    - New data storage can be added without touching any application code or schema definitions
    - The tables still exist in the same tablespace, the tablespace just got larger.
    - Without this abstraction, every physical storage expansion would require schema changes
  - Separation of different classes of data
    - Application data, indexes, temporary sort data
  - Performance placement
    - Different tablespaces can be backed by storage with different performance characteristics

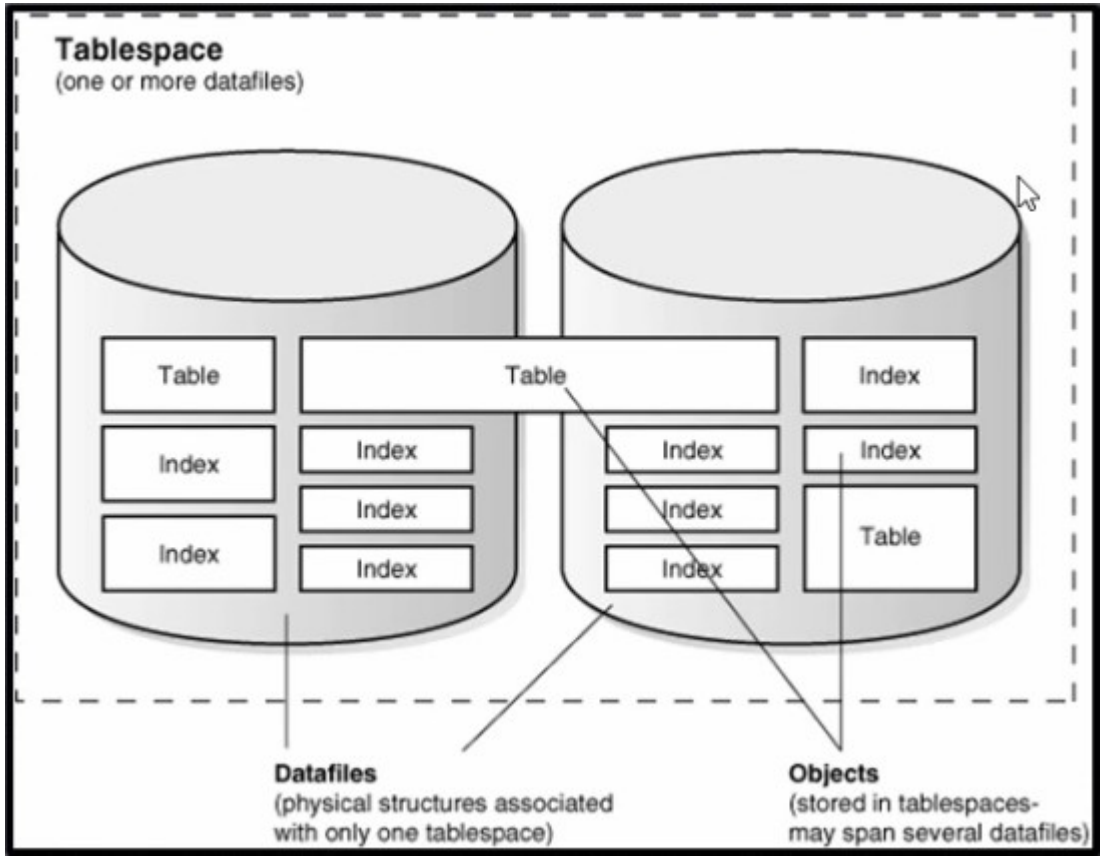


Image Credit: https://sql.standout-dev.com/learn-oracle-sql/basic-concepts/basic-oracle-database-concepts-and-architecture/

Oracle Storage Hierarchy

- Tablespace
  - Availability and backup control
    - A tablespace can be taken offline independently, backed up independently, or made read-only independently
    - In a large database it is common to make historical data tablespaces read-only once the data will never change
    - Partitioned tables can be configured so that different partitions live in different tablespaces
    - Allows the DBA to take old partitions offline or archive them without affecting current data.

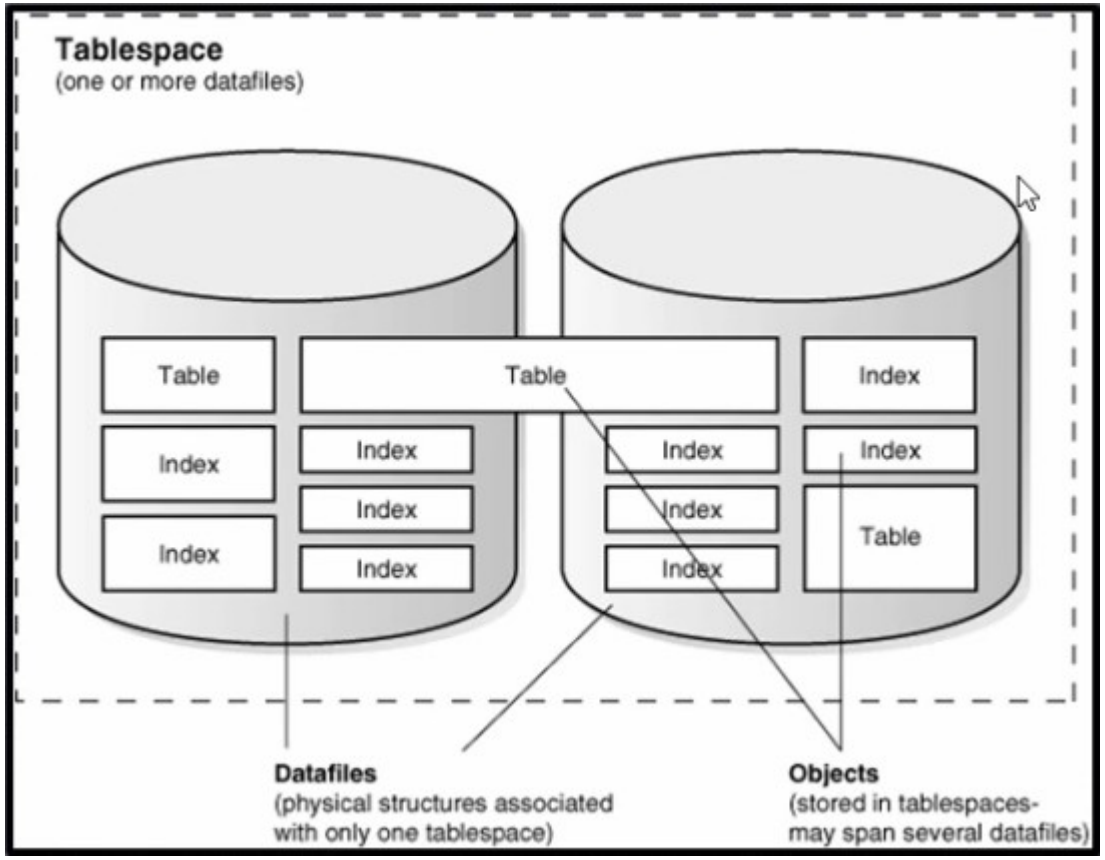


Image Credit: https://sql.standout-dev.com/learn-oracle-sql/basic-concepts/basic-oracle-database-concepts-and-architecture/

Oracle Storage Hierarchy

- Segment
  - Storage allocation for one specific database object
  - Every table has one segment
  - Every index has its own separate segment
  - The segment holds all the blocks that belong to that object
  - When an execution plan reports I/O costs, it is measuring work done against segments

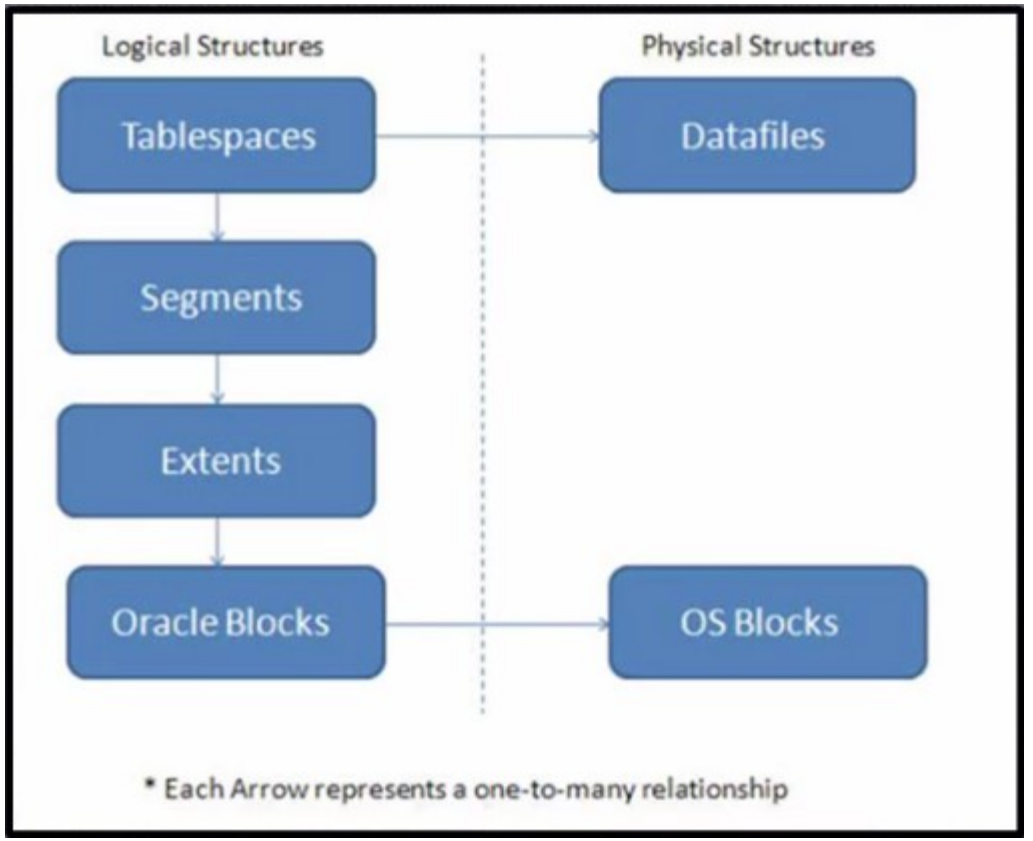


Image Credit: https://sql.standout-dev.com/learn-oracle-sql/basic-concepts/basic-oracle-database-concepts-and-architecture/

Oracle Storage Hierarchy

- Extent
  - An extent is a contiguous set of blocks allocated to a segment in a single allocation
  - When a table grows beyond its current allocated space, Oracle adds a new extent to its segment
  - Extents do not have to be contiguous with each other on disk
    - a segment can consist of many extents scattered across the tablespace's data files
  - The number and size of extents in a segment has no effect on query correctness, only on storage management

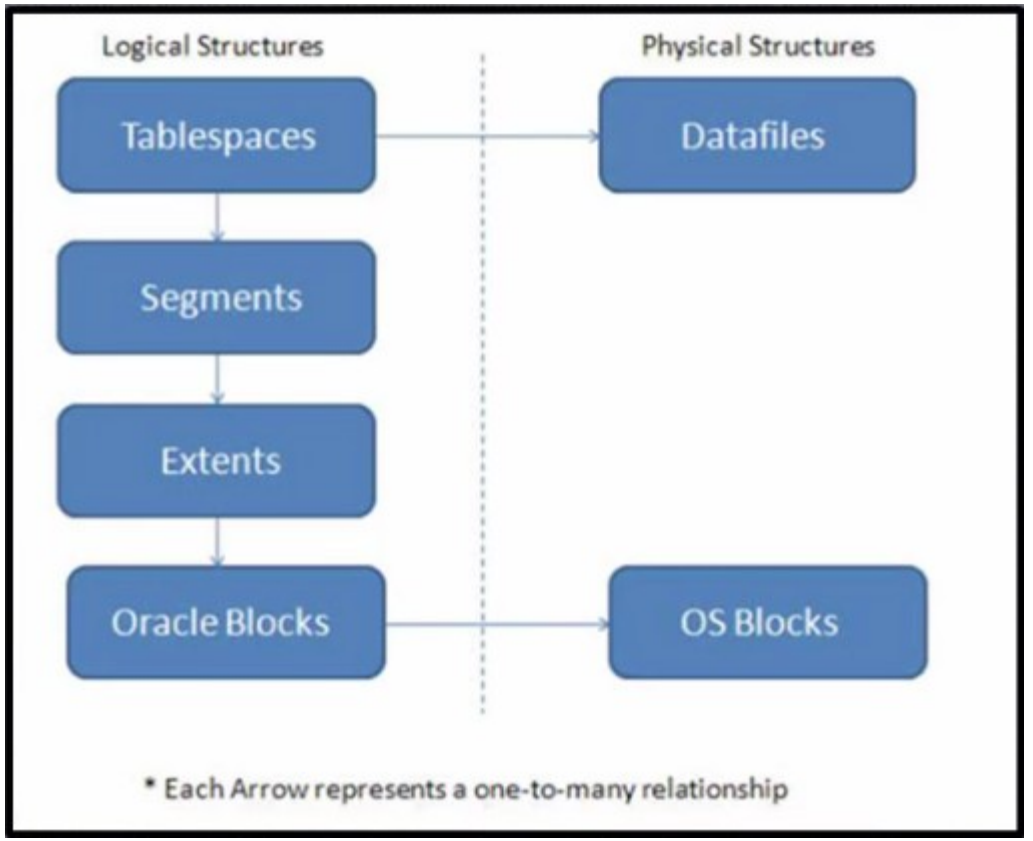


Image Credit: https://sql.standout-dev.com/learn-oracle-sql/basic-concepts/basic-oracle-database-concepts-and-architecture/

Relational Tables

- ROWID: Oracle's physical address
  - Every row in a heap table has a unique ROWID
    - Its physical location made up of block, row slot, data file
  - Secondary indexes store the ROWID of indexed rows to enable fast table access
  - ROWID is stable unless a row is deleted and re-inserted
    - It is not a substitute for a primary key
  - How index-to-table access works
    - The index stores a pointer to the physical row

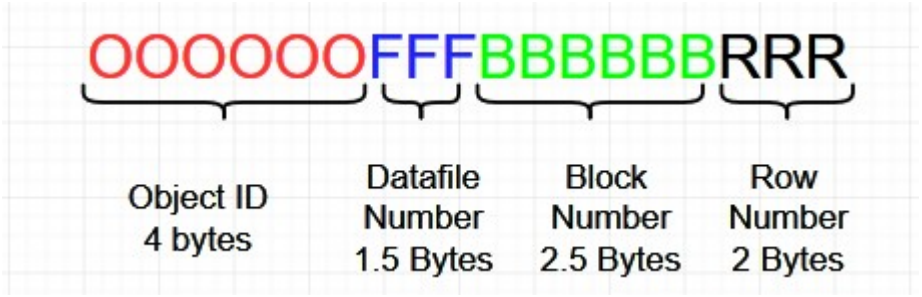


Image Credit: https://www.slideserve.com/jennis/lecture-4-oracle-tablespaces

Data Types

Data Types

- Data types
  - Declares what kind of value a column is permitted to hold
  - Choosing the wrong data type
    - Can potentially waste storage
    - Changes what the column can express
  - Every data type choice is a modeling decision that encodes a business rule
    - Specifically, what can be done with this data from a logical perspective

Type	Use When	Avoid When
NUMBER(p, s)	Exact numeric values; monetary amounts; quantities	Approximate calculations are acceptable (use BINARY_DOUBLE instead)
VARCHAR2(n)	Variable-length text up to 4000 bytes	Storing structured data as a string (amounts, dates, codes)
CHAR(n)	Fixed-length codes where all values are the same length (e.g., a 2-character country code)	General text -- trailing space padding causes subtle comparison bugs
DATE	Date and time to the nearest second	Sub-second precision is needed
TIMESTAMP(p)	Date and time with fractional seconds	Only date-level precision is needed (DATE is sufficient and simpler)
TIMESTAMP WITH TIME ZONE	Timestamps from multiple time zones	All data originates from a single time zone
NUMBER GENERATED AS IDENTITY	Surrogate primary keys	A natural key is stable, unique, and meaningful
CLOB	Text exceeding 4000 bytes	Text that will be searched, sorted, or compared in SQL (use VARCHAR2)
BLOB	Binary data: images, documents, encrypted content	Structured data that should be in columns

Data Types

- A few common type mistakes
  - Storing a monetary amount as VARCHAR2
    - Cannot sum, cannot sort correctly, cannot range query
  - Storing a date as VARCHAR2
    - Cannot range query, locale-dependent format bugs, no validation
  - Using CHAR for variable-length text
    - Trailing space padding breaks equality comparisons unless trimmed
  - Using TIMESTAMP WITH TIME ZONE carelessly
    - Time zone arithmetic is subtle and error-prone

Type	Use When	Avoid When
NUMBER(p, s)	Exact numeric values; monetary amounts; quantities	Approximate calculations are acceptable (use BINARY_DOUBLE instead)
VARCHAR2(n)	Variable-length text up to 4000 bytes	Storing structured data as a string (amounts, dates, codes)
CHAR(n)	Fixed-length codes where all values are the same length (e.g., a 2-character country code)	General text -- trailing space padding causes subtle comparison bugs
DATE	Date and time to the nearest second	Sub-second precision is needed
TIMESTAMP(p)	Date and time with fractional seconds	Only date-level precision is needed (DATE is sufficient and simpler)
TIMESTAMP WITH TIME ZONE	Timestamps from multiple time zones	All data originates from a single time zone
NUMBER GENERATED AS IDENTITY	Surrogate primary keys	A natural key is stable, unique, and meaningful
CLOB	Text exceeding 4000 bytes	Text that will be searched, sorted, or compared in SQL (use VARCHAR2)
BLOB	Binary data: images, documents, encrypted content	Structured data that should be in columns

Data Types

- LOBs
  - A standard VARCHAR2 column holds up to 4,000 bytes
    - 32,767 bytes with extended data types enabled
  - Many real-world data items exceed this limit
    - Long-form text, documents, images, audio, encrypted payloads
  - Storing large values inline in a table row creates problems
    - Rows become very wide, blocks fill quickly
    - table scans read enormous amounts of data even when the large column is not needed

Type	Stores	Typical Use Cases
CLOB (Character Large Object)	Text data in the database character set	Long-form text, XML documents, JSON payloads, log content, configuration data
NCLOB (National Character Large Object)	Text data in the national character set (Unicode)	Same as CLOB but required when the data contains characters outside the database character set
BLOB (Binary Large Object)	Raw binary data with no character set interpretation	Images, PDFs, office documents, audio files, video, encrypted content, serialized binary formats

Data Types

- LOBs
  - Oracle solves this with LOB storage
    - Large object values are stored separately from the table row, with only a locator pointer remaining in the row itself
  - LOB values are stored out-of-line in a separate LOB segment associated with the table
  - The table row contains only a LOB locator
    - A small pointer to where the actual value lives
  - Reading a LOB requires a separate I/O to the LOB segment
    - It is not retrieved as part of a normal table block read
  - Small LOB values <~ 4,000 bytes
    - Can be stored inline in the row itself using the ENABLE STORAGE IN ROW option, avoiding the extra I/O for small values

Type	Stores	Typical Use Cases
CLOB (Character Large Object)	Text data in the database character set	Long-form text, XML documents, JSON payloads, log content, configuration data
NCLOB (National Character Large Object)	Text data in the national character set (Unicode)	Same as CLOB but required when the data contains characters outside the database character set
BLOB (Binary Large Object)	Raw binary data with no character set interpretation	Images, PDFs, office documents, audio files, video, encrypted content, serialized binary formats

Data Types

- LOBs
  - Oracle has two LOB storage implementations
  - BasicFile is the legacy implementation, available in all versions
  - SecureFile is the modern implementation
    - Significantly faster, supports compression, deduplication, and encryption at the LOB level
  - SecureFile is the correct choice for all new development
  - BasicFile exists only for backward compatibility

Type	Stores	Typical Use Cases
CLOB (Character Large Object)	Text data in the database character set	Long-form text, XML documents, JSON payloads, log content, configuration data
NCLOB (National Character Large Object)	Text data in the national character set (Unicode)	Same as CLOB but required when the data contains characters outside the database character set
BLOB (Binary Large Object)	Raw binary data with no character set interpretation	Images, PDFs, office documents, audio files, video, encrypted content, serialized binary formats

Data Types

- CLOBs
  - The text content is genuinely variable and unbounded in length
    - A notes field that could be a sentence or ten paragraphs, an XML response from an external API, a full document body
  - The content will be read and written as a whole unit, not searched or sorted in SQL
  - If the content needs to be searched
    - Oracle Text indexes can be built on CLOB columns, enabling full-text search
    - But this requires explicit setup and has its own performance characteristics

Type	Stores	Typical Use Cases
CLOB (Character Large Object)	Text data in the database character set	Long-form text, XML documents, JSON payloads, log content, configuration data
NCLOB (National Character Large Object)	Text data in the national character set (Unicode)	Same as CLOB but required when the data contains characters outside the database character set
BLOB (Binary Large Object)	Raw binary data with no character set interpretation	Images, PDFs, office documents, audio files, video, encrypted content, serialized binary formats

Data Types

- When NOT to use CLOBs
  - Storing structured data as text
    - Status codes, amounts, dates, identifiers
    - These belong in properly typed columns
  - Storing JSON when Oracle 21c or later is available
    - The native JSON data type is purpose-built for JSON storage
    - Querying outperforms CLOB-stored JSON significantly
  - Storing text that will be compared, sorted, or used in WHERE clause equality predicates
    - VARCHAR2 is far more efficient for this

Type	Stores	Typical Use Cases
CLOB (Character Large Object)	Text data in the database character set	Long-form text, XML documents, JSON payloads, log content, configuration data
NCLOB (National Character Large Object)	Text data in the national character set (Unicode)	Same as CLOB but required when the data contains characters outside the database character set
BLOB (Binary Large Object)	Raw binary data with no character set interpretation	Images, PDFs, office documents, audio files, video, encrypted content, serialized binary formats

Data Types

- When to use BLOBs
  - Binary content that is genuinely part of the application's data model
    - A document management system storing PDFs
    - An image catalog storing photos
    - An audit system storing digitally signed records
  - Encrypted data
    - When column-level encryption is applied to sensitive values
    - the encrypted output is binary and belongs in a BLOB
  - Content where the database is the authoritative store and retrieval always goes through the application layer

Type	Stores	Typical Use Cases
CLOB (Character Large Object)	Text data in the database character set	Long-form text, XML documents, JSON payloads, log content, configuration data
NCLOB (National Character Large Object)	Text data in the national character set (Unicode)	Same as CLOB but required when the data contains characters outside the database character set
BLOB (Binary Large Object)	Raw binary data with no character set interpretation	Images, PDFs, office documents, audio files, video, encrypted content, serialized binary formats

Data Types

- When NOT to use BLOBs
  - Large media files like video, high-resolution images at scale
    - The database is rarely the right store for these
    - Object storage systems are purpose-built for this workload and far more cost-effective
  - Files that will be served directly to end users via a web server
    - A file system or object store with a CDN is more efficient for this pattern
  - When the volume of binary data is large enough that it will dominate storage costs and backup times
    - LOB segments can grow very large and complicate DBA operations

Type	Stores	Typical Use Cases
CLOB (Character Large Object)	Text data in the database character set	Long-form text, XML documents, JSON payloads, log content, configuration data
NCLOB (National Character Large Object)	Text data in the national character set (Unicode)	Same as CLOB but required when the data contains characters outside the database character set
BLOB (Binary Large Object)	Raw binary data with no character set interpretation	Images, PDFs, office documents, audio files, video, encrypted content, serialized binary formats

Data Types

- The architectural question to ask first
  - Does this data need to be queried, filtered, or joined in SQL?
    - If yes, it probably belongs in structured columns, not a LOB
  - Is the database the right store at all?
    - For large binary content at scale, a dedicated object store (with a URL or key stored in the database) is often a better architecture than a BLOB column

Type	Stores	Typical Use Cases
CLOB (Character Large Object)	Text data in the database character set	Long-form text, XML documents, JSON payloads, log content, configuration data
NCLOB (National Character Large Object)	Text data in the national character set (Unicode)	Same as CLOB but required when the data contains characters outside the database character set
BLOB (Binary Large Object)	Raw binary data with no character set interpretation	Images, PDFs, office documents, audio files, video, encrypted content, serialized binary formats

Oracle Data Types

- Relational databases
  - Were originally designed to store structured, scalar values
    - Numbers, short strings, and dates
    - Because that was the nature of business data when the relational model was introduced
  - Modern enterprise applications deal with data that does not fit neatly into scalar columns
    - Geographic coordinates that need distance and containment calculations
    - Documents that need to be queried by their internal structure
    - Time durations that are meaningful values in their own right rather than differences between two dates

Type	Category	What It Stores	Use When	Avoid When
SDO_GEOMETRY	Spatial	Points, lines, polygons, and geographic shapes	Any query involving location, proximity, distance, or area	No spatial queries are needed -- store lat/long as two NUMBER columns instead
XMLType	Semi-structured	Native XML documents with XQuery/XPath support	XML integration with external systems and standards-based message formats	New development where JSON is an option -- JSON tooling is simpler
JSON	Semi-structured	Native binary JSON with field-level access	Storing JSON payloads that will be queried at the field level (Oracle 21c and later)	Simple structured data that belongs in typed columns
INTERVAL YEAR TO MONTH	Interval	A duration expressed in years and months	Storing a length of time as a first-class value, such as a contract term of 2 years 6 months	A specific end date is what matters -- store the date instead
INTERVAL DAY TO SECOND	Interval	A duration expressed in days, hours, minutes, and seconds	Elapsed time, SLA durations, response times	The duration will never be queried or calculated in SQL
Virtual column	Derived	An expression computed from other columns, optionally indexed	A derived value that must be queryable and indexable without being physically stored	The expression is expensive -- a materialized view may be more appropriate
UROWID	Internal	A portable row address that works across heap tables, IOTs, and remote tables	Temporary row references in PL/SQL processing	Application primary keys -- not durable across table reorganizations
ANYDATA	Generic	A value of any Oracle type	Oracle internal infrastructure and highly generic framework code	Application schemas -- almost always signals a data modeling problem

Oracle Data Types

- Handling these data types outside the database in application code
  - Means the database cannot index them, enforce constraints on them, or optimize queries against them
  - Pushing the logic into the database engine makes it available to every consumer
  - Allows the optimizer to reason about it

Type	Category	What It Stores	Use When	Avoid When
SDO_GEOMETRY	Spatial	Points, lines, polygons, and geographic shapes	Any query involving location, proximity, distance, or area	No spatial queries are needed -- store lat/long as two NUMBER columns instead
XMLType	Semi-structured	Native XML documents with XQuery/XPath support	XML integration with external systems and standards-based message formats	New development where JSON is an option -- JSON tooling is simpler
JSON	Semi-structured	Native binary JSON with field-level access	Storing JSON payloads that will be queried at the field level (Oracle 21c and later)	Simple structured data that belongs in typed columns
INTERVAL YEAR TO MONTH	Interval	A duration expressed in years and months	Storing a length of time as a first-class value, such as a contract term of 2 years 6 months	A specific end date is what matters -- store the date instead
INTERVAL DAY TO SECOND	Interval	A duration expressed in days, hours, minutes, and seconds	Elapsed time, SLA durations, response times	The duration will never be queried or calculated in SQL
Virtual column	Derived	An expression computed from other columns, optionally indexed	A derived value that must be queryable and indexable without being physically stored	The expression is expensive -- a materialized view may be more appropriate
UROWID	Internal	A portable row address that works across heap tables, IOTs, and remote tables	Temporary row references in PL/SQL processing	Application primary keys -- not durable across table reorganizations
ANYDATA	Generic	A value of any Oracle type	Oracle internal infrastructure and highly generic framework code	Application schemas -- almost always signals a data modeling problem

Oracle Data Types

- Oracle's extended types
  - Decades of enterprise use cases accumulating in a single product
  - Each type was added because a large class of Oracle customers had a recurring need
    - Were being solving badly with scalar columns
  - For example
    - Storing lat/long as two NUMBERs and computing distances in application code
    - Storing XML as CLOB and parsing it in Java
    - Storing JSON as text and deserializing the whole document to read one field

Type	Category	What It Stores	Use When	Avoid When
SDO_GEOMETRY	Spatial	Points, lines, polygons, and geographic shapes	Any query involving location, proximity, distance, or area	No spatial queries are needed -- store lat/long as two NUMBER columns instead
XMLType	Semi-structured	Native XML documents with XQuery/XPath support	XML integration with external systems and standards-based message formats	New development where JSON is an option -- JSON tooling is simpler
JSON	Semi-structured	Native binary JSON with field-level access	Storing JSON payloads that will be queried at the field level (Oracle 21c and later)	Simple structured data that belongs in typed columns
INTERVAL YEAR TO MONTH	Interval	A duration expressed in years and months	Storing a length of time as a first-class value, such as a contract term of 2 years 6 months	A specific end date is what matters -- store the date instead
INTERVAL DAY TO SECOND	Interval	A duration expressed in days, hours, minutes, and seconds	Elapsed time, SLA durations, response times	The duration will never be queried or calculated in SQL
Virtual column	Derived	An expression computed from other columns, optionally indexed	A derived value that must be queryable and indexable without being physically stored	The expression is expensive -- a materialized view may be more appropriate
UROWID	Internal	A portable row address that works across heap tables, IOTs, and remote tables	Temporary row references in PL/SQL processing	Application primary keys -- not durable across table reorganizations
ANYDATA	Generic	A value of any Oracle type	Oracle internal infrastructure and highly generic framework code	Application schemas -- almost always signals a data modeling problem

Oracle Data Types

- Each Oracle-specific type
  - Comes with a corresponding set of functions, operators, and indexing capabilities
    - These only make sense for that data type
    - SDO_GEOMETRY is only useful because spatial indexes and spatial operators exist alongside it
    - The JSON type is only useful because JSON path expressions and JSON-specific indexes exist to query into it
  - The type and its supporting infrastructure are designed together as a unit

Type	Category	What It Stores	Use When	Avoid When
SDO_GEOMETRY	Spatial	Points, lines, polygons, and geographic shapes	Any query involving location, proximity, distance, or area	No spatial queries are needed -- store lat/long as two NUMBER columns instead
XMLType	Semi-structured	Native XML documents with XQuery/XPath support	XML integration with external systems and standards-based message formats	New development where JSON is an option -- JSON tooling is simpler
JSON	Semi-structured	Native binary JSON with field-level access	Storing JSON payloads that will be queried at the field level (Oracle 21c and later)	Simple structured data that belongs in typed columns
INTERVAL YEAR TO MONTH	Interval	A duration expressed in years and months	Storing a length of time as a first-class value, such as a contract term of 2 years 6 months	A specific end date is what matters -- store the date instead
INTERVAL DAY TO SECOND	Interval	A duration expressed in days, hours, minutes, and seconds	Elapsed time, SLA durations, response times	The duration will never be queried or calculated in SQL
Virtual column	Derived	An expression computed from other columns, optionally indexed	A derived value that must be queryable and indexable without being physically stored	The expression is expensive -- a materialized view may be more appropriate
UROWID	Internal	A portable row address that works across heap tables, IOTs, and remote tables	Temporary row references in PL/SQL processing	Application primary keys -- not durable across table reorganizations
ANYDATA	Generic	A value of any Oracle type	Oracle internal infrastructure and highly generic framework code	Application schemas -- almost always signals a data modeling problem

Oracle Data Types

- Spatial data: SDO_GEOMETRY
  - Included in Enterprise Edition
    - Provides the SDO_GEOMETRY type for storing geometric and geographic data
    - Stores points, lines, polygons, and complex geometric shapes
  - Supports spatial indexes (R-tree indexes) that enable efficient proximity queries
    - "Find all locations within 5 kilometers of this point"
  - Supports standard spatial operations
    - Intersection, containment, distance calculation, area computation
    - Used in applications that work with maps, routing, geofencing, asset tracking, and territory management
    - Queries use the SDO_RELATE, SDO_WITHIN_DISTANCE, and SDO_GEOM function families

```
CREATE TABLE branch_locations (
    branch_id   NUMBER PRIMARY KEY,
    branch_name VARCHAR2(100),
    location    SDO_GEOMETRY  -- stores latitude/longitude as a point geometry
);
```

Oracle Data Types

- XML: XMLType
  - A dedicated type for storing and querying XML documents natively
  - Oracle stores XMLType content in an optimized binary format internally
    - Object-relational or binary XML storage
    - Supports XQuery and XPath expressions directly in SQL for querying into the XML structure
    - XML DB (included with Oracle) provides a repository model and HTTP access to XML content
  - Largely superseded by JSON in modern applications but remains common in legacy systems and standards-based message formats (SWIFT, HL7, SOAP)

```
SELECT x.extract('/Order/CustomerID/text()').getStringVal()
FROM   orders_xml x
WHERE  x.xml_data.existsNode('/Order[@status="PENDING"]') = 1;
```

Oracle Data Types

- JSON
  - Native binary storage format optimized for field-level access without parsing the full document
    - Supports SQL/JSON path expressions for querying into document structure
    - Allows JSON-specific indexes on individual fields within the document
    - Enforces optional JSON schema validation at the column level
  - Significantly faster than CLOB-stored JSON for field-level queries and updates

```
CREATE TABLE events (
    event_id   NUMBER PRIMARY KEY,
    payload    JSON,
    CONSTRAINT chk_payload CHECK (payload IS JSON)
);

-- Query a field inside the JSON document
SELECT e.payload.customer_id, e.payload.event_type
FROM   events e
WHERE  e.payload.status = 'ACTIVE';
```

Oracle Data Types

- Interval types
  - Store a duration rather than a point in time
  - INTERVAL YEAR TO MONTH
    - Stores a number of years and months for calendar-based durations
  - INTERVAL DAY TO SECOND
    - Stores a number of days, hours, minutes, and seconds with fractional precision
  - Useful for storing
    - SLAs, durations, time-to-completion
    - age calculations where the interval itself is the meaningful value rather than a derived calculation

```
// Store subscription length as a duration, not a computed end date
subscription_length  INTERVAL YEAR TO MONTH
    // e.g. INTERVAL '2-6' YEAR TO MONTH = 2 years 6 months

// Store response time as a duration
response_time        INTERVAL DAY TO SECOND
    // e.g. INTERVAL '0 04:32:17.5' DAY TO SECOND
```

Oracle Data Types

- Virtual columns
  - Type-adjacent feature
  - A column defined by an expression rather than a stored value
  - Oracle evaluates the expression on the fly when the column is queried
  - Can be indexed
    - Computes the result for all rows when the index is created
    - This is stored in a materialized index structure
    - Index is recomputed when data is added or changed

```
ALTER TABLE employees ADD (
    annual_salary NUMBER GENERATED ALWAYS AS (monthly_salary * 12) VIRTUAL
);

-- annual_salary can now be queried and indexed without being stored
CREATE INDEX idx_emp_annual_sal ON employees (annual_salary);
```

Oracle Data Types

- ANYDATA, ANYTYPE, ANYDATASET
  - Generic container types that can hold a value of any Oracle type
  - Used primarily in Oracle's own internal infrastructure and in highly generic framework code
  - Rarely appropriate in application schemas
    - Use is usually a sign that the data model has not been thought through
  - "We need a column that could hold anything"
    - Almost always the solution is to redesign the model, not to use ANYDATA

```
ALTER TABLE employees ADD (
    annual_salary NUMBER GENERATED ALWAYS AS (monthly_salary * 12) VIRTUAL
);

-- annual_salary can now be queried and indexed without being stored
CREATE INDEX idx_emp_annual_sal ON employees (annual_salary);
```

Keys

Key Types

- Primary key guarantees
  - Defined in the relational model
  - Uniqueness
    - No two rows in the table share the same primary key value
  - Stability
    - The primary key value does not change after the row is inserted
  - Non-nullability
    - Every row must have a primary key value with no exceptions

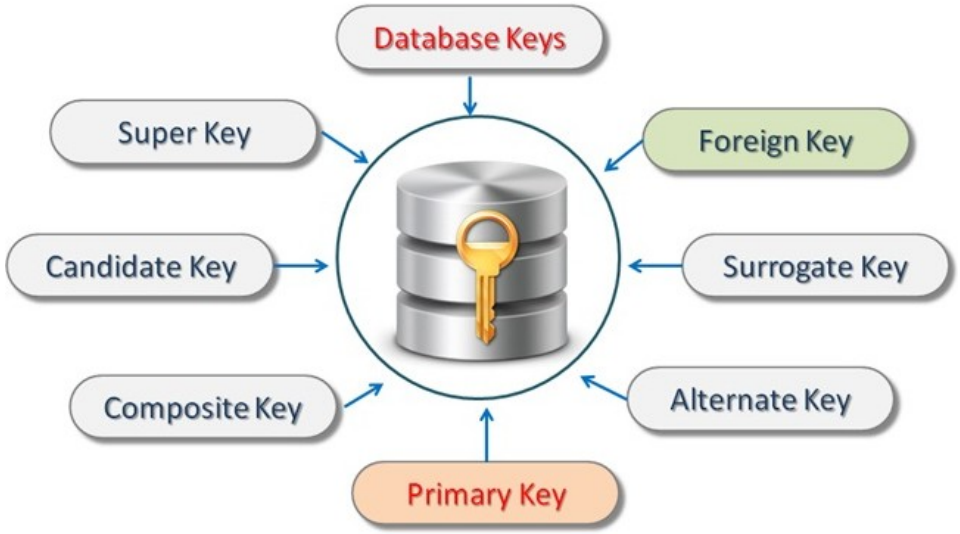


Image Credit: https://www.learncomputerscienceonline.com/database-keys/

Key Types

- Natural keys
  - An attribute that already exists in the domain and is inherently unique
    - For example, a product barcode, an email address, a national identifier number
  - Advantage
    - The key carries meaning; a join result is self-documenting
- The problem with natural keys
  - Real-world identifiers are frequently less stable than they appear at design time
    - For example; Email addresses change, product barcodes get reassigned, codes get reformatted
    - A natural key that changes requires updating every table that references it as a foreign key

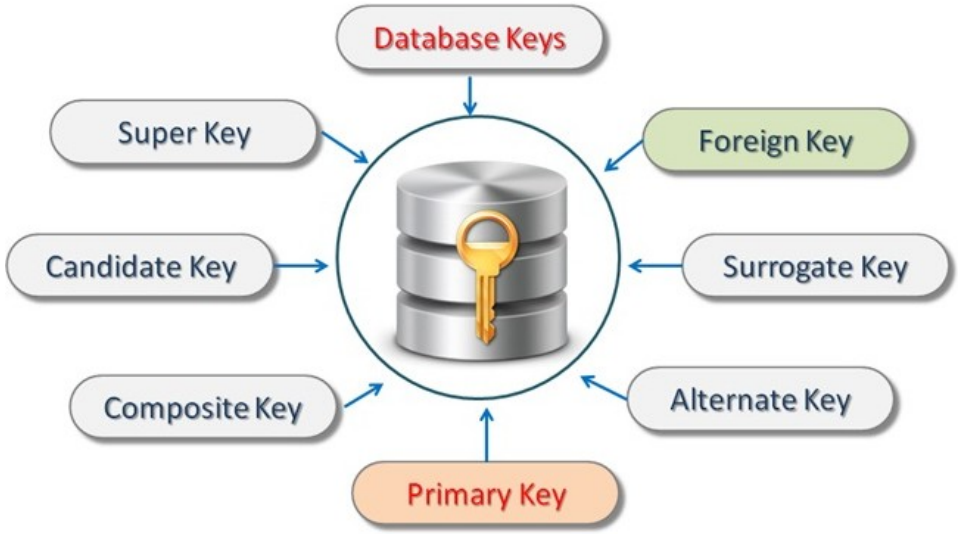


Image Credit: https://www.learncomputerscienceonline.com/database-keys/

Key Types

- Composite keys
  - Used for natural keys when no single attribute is unique
  - Candidate keys are combinations of attributes that provide a unique identifier
    - For example full name and birth date
    - All the columns together are a candidate key
    - The primary key is the smallest candidate key
  - Doesn't solve the problem
    - Individual parts of the composite key can still change
  - Usually results in wide keys
    - Slows down access and computation

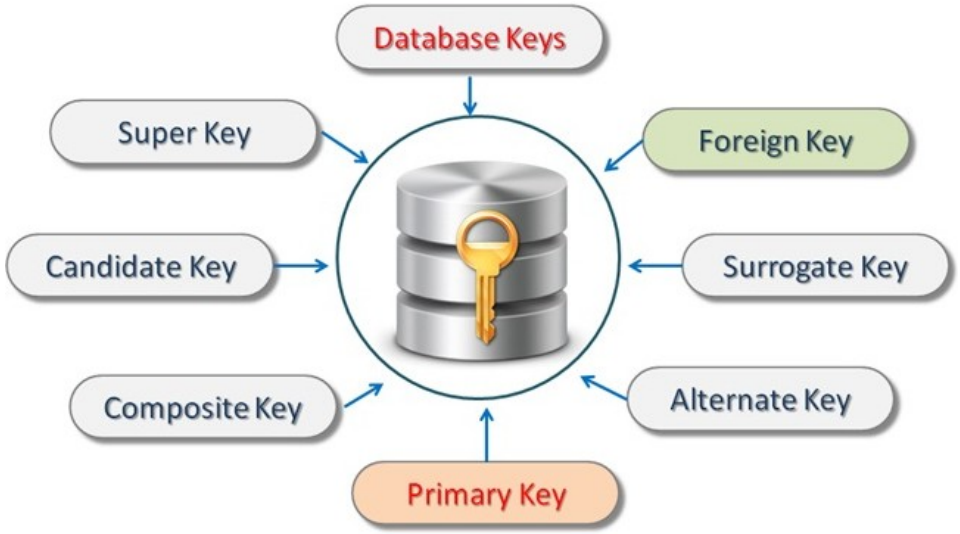


Image Credit: https://www.learncomputerscienceonline.com/database-keys/

Key Types

- Natural unique keys
  - These are domain values
  - Used as a unique identifier for domain objects
    - For example; employee ID, SSN,
    - Generated withing the domain
- Often used as primary keys
  - The uniqueness and integrity is now dependent on the issuing authority
  - May fail completeness or uniqueness
    - No guarantee every entity is identified
    - No guarantee the issuing authority is ensuring uniqueness

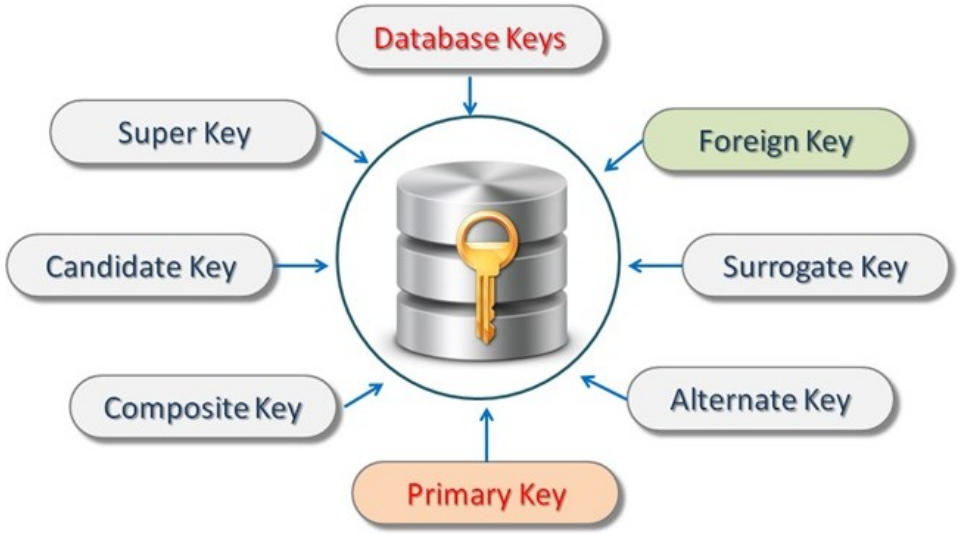


Image Credit: https://www.learncomputerscienceonline.com/database-keys/

Key Types

- The surrogate key
  - Is stable by design
  - It has no real-world meaning to change
  - The natural identifier like project_code
    - Retained as a UNIQUE constraint for lookups and display
  - Foreign keys reference project_id: narrow, stable, integer joins
- This pattern separates
  - Identity in the database
  - Identification in the domain

```
CREATE TABLE projects (
        project_id   NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
        project_code VARCHAR2(20)  NOT NULL UNIQUE,  -- natural key retained as a unique constraint
project_name VARCHAR2(200) NOT NULL,
status       VARCHAR2(20)  NOT NULL
);
```

Key Types

- Practical recommendation
  - Use surrogate keys for all relationships and internal joins
  - Retain the natural key as a UNIQUE constraint
    - Stable, narrow foreign keys for join performance
    - Natural identifier for display, API responses, and user-facing lookups. The code example demonstrates this pattern explicitly
  - For example, employee ID as primary key
    - Employee leaves and returns with a new ID
    - Company merges with another that has overlapping ID
    - Identifier format changes with a regulatory update

```
CREATE TABLE projects (
        project_id   NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
        project_code VARCHAR2(20)  NOT NULL UNIQUE,  -- natural key retained as a unique constraint
project_name VARCHAR2(200) NOT NULL,
status       VARCHAR2(20)  NOT NULL
);
```

Key Types

- Surrogate keys types
  - NUMBER
    - The traditional choice, typically a plain integer generated from a sequence
    - Compact, fast to compare, fast to join
    - B-tree index on it is narrow and efficient
    - This is the a common choice in Oracle systems.

```
-- Approach 1: explicit sequence (pre-Oracle 12c style, still widely used)
CREATE SEQUENCE seq_projects START WITH 1 INCREMENT BY 1 NOCACHE NOCYCLE;

CREATE TABLE projects (
    project_id   NUMBER        NOT NULL,
    project_name VARCHAR2(200) NOT NULL,
    CONSTRAINT pk_projects PRIMARY KEY (project_id)
);

-- Populated at insert time via the sequence
INSERT INTO projects (project_id, project_name)
VALUES (seq_projects.NEXTVAL, 'Atlas Redesign');
```

Key Types

- Surrogate keys types
  - NUMBER GENERATED AS IDENTITY
    - Introduced in Oracle 12c
    - Combines the NUMBER column with an implicit sequence in a single declaration
    - Functionally identical to a sequence-populated NUMBER but simpler to define and maintain.
  - INTEGER
    - Maps to NUMBER(38) internally in Oracle
    - Functionally the same as NUMBER for surrogate key purposes

```
-- Approach 2: GENERATED AS IDENTITY (Oracle 12c and later -- preferred)
CREATE TABLE projects (
    project_id   NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    project_name VARCHAR2(200) NOT NULL
);

-- No value supplied for project_id -- Oracle generates it automatically
INSERT INTO projects (project_name) VALUES ('Atlas Redesign');
```

```
-- Approach 3: GENERATED BY DEFAULT AS IDENTITY
-- Allows explicit value override when needed (e.g. during data migration)
CREATE TABLE projects (
                        project_id   NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
                        project_name VARCHAR2(200) NOT NULL
);

-- Oracle generates the value normally
INSERT INTO projects (project_name) VALUES ('Atlas Redesign');

-- But an explicit value can be supplied when necessary
INSERT INTO projects (project_id, project_name) VALUES (9999, 'Legacy Migration Row');
```

Key Types

- Surrogate keys types
  - RAW(16) populated with SYS_GUID()
    - Stores a 16-byte globally unique identifier (a GUID/UUID)
    - Unique across databases and systems
    - Useful when rows are generated in multiple systems and merged
    - The index fragmentation problem was improved in 18c with an internal algorithm change
    - Prior to that, the randomness of the generated data led to poorly clustered indexes
    - 16-byte key is also eight times wider than a 4-byte integer, which increases the size of every index that references it as a foreign key

```
-- Table definition using RAW(16) as a surrogate key
CREATE TABLE projects (
    project_id   RAW(16)       DEFAULT SYS_GUID() PRIMARY KEY,
    project_name VARCHAR2(200) NOT NULL,
    status       VARCHAR2(20)  NOT NULL
);

-- Insert without supplying a key -- SYS_GUID() fires from the column default
INSERT INTO projects (project_name, status)
VALUES ('Atlas Redesign', 'ACTIVE');

-- Insert with an explicit GUID when the value must be known before insert
-- (for example, to set up a foreign key in a child table in the same transaction)
INSERT INTO projects (project_id, project_name, status)
VALUES (SYS_GUID(), 'Mercury Integration', 'PLANNED');

-- Querying by primary key -- note the hex literal syntax
SELECT project_name, status
FROM   projects
WHERE  project_id = HEXTORAW('A1B2C3D4E5F6A1B2C3D4E5F6A1B2C3D4');
```

Relationships

Relationships

- Recall from Chen's framework
  - At Level 1
    - A relationship is an association among entities
    - "An employee is assigned to a project"
  - At Level 2
    - The relationship has a formal structure
    - Which entity sets participate
    - What the cardinality is
    - what attributes the relationship itself carries

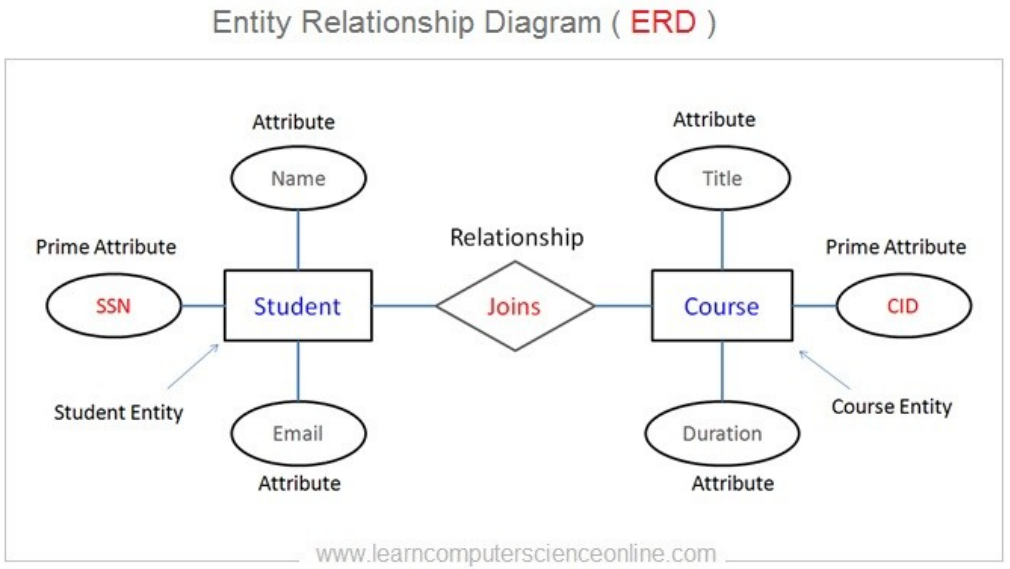


Image Credit: https://www.learncomputerscienceonline.com/entity-relationship-diagram/

Relationships

- Recall from Chen's framework
  - At Level 3
    - The relationship must be expressed using only tables and column values
    - There are no pointers or links in the relational model
- Relational model
  - One-to-many relationship
  - The many table has the one table's key as a foreign key

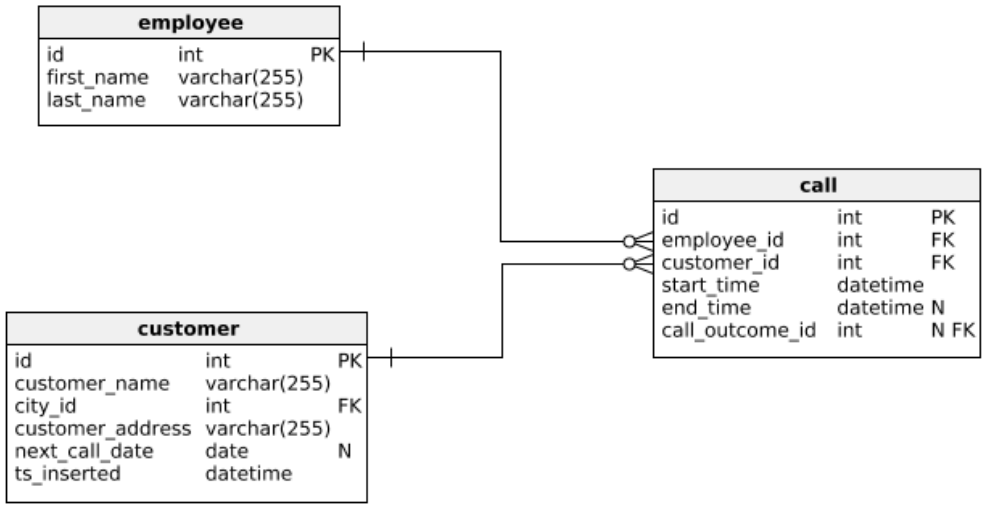


Image Credit: https://www.sqlshack.com/learn-sql-types-of-relations/

Relationships

- Relational model
  - Many-to-many relationship
  - Represented by introducing a junction table that holds the primary keys of both participating entities
  - All relationships are expressed as column values
  - The join operation reconstructs the relationship at query time

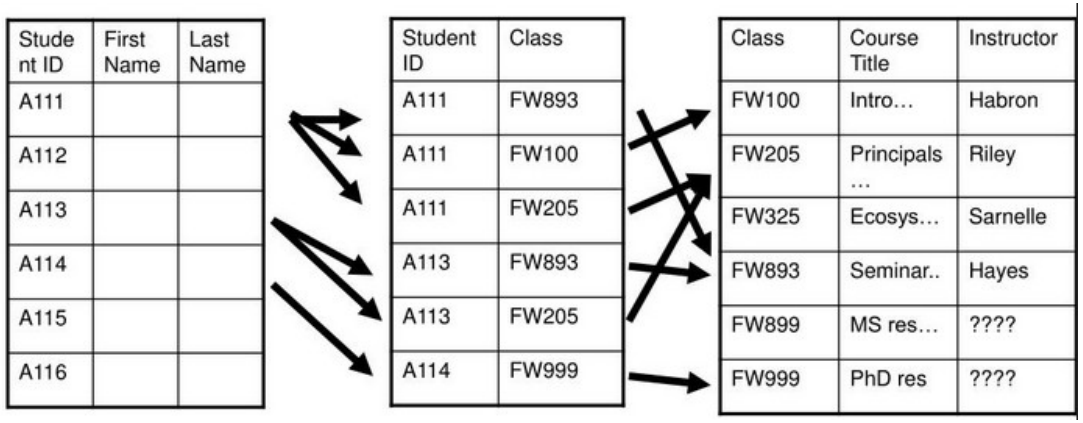
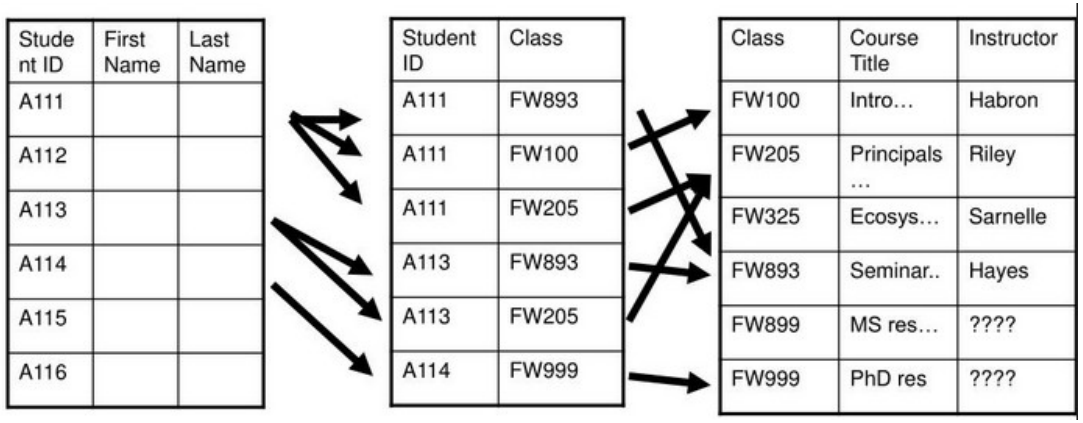


Image Credit: https://www.slideserve.com/tyrone/table-relationships

Relationships

- Why this design is used
  - Any relationship can be traversed in any direction without redesigning the schema
  - New access patterns (new queries) do not require new physical structures, only new SQL
- Pitfalls
  - The cardinality must be correct at the domain level before it is implemented
  - A one-to-many implemented as many-to-many is wasteful but survivable
  - A many-to-many implemented as one-to-many is a structural error that corrupts data
  - Cardinality errors discovered after data is loaded are expensive, they require schema migration and data transformation



Stude nt ID	First Name	Last Name
A111		
A112		
A113		
A114		
A115		
A116		

Student ID	Class
A111	FW893
A111	FW100
A111	FW205
A113	FW893
A113	FW205
A114	FW999

Class	Course Title	Instructor
FW100	Intro…	Habron
FW205	Principals …	Riley
FW325	Ecosys…	Sarnelle
FW893	Seminar..	Hayes
FW899	MS res…	????
FW999	PhD res	????

Image Credit: https://www.slideserve.com/tyrone/table-relationships

Relationships in Oracle

- One-to-many
  - The most common case
  - The one-to-many example realizes the level 1 relationship
    - "An employee belongs to a department"
  - One department, many employees
    - The department_id foreign key in the employees table is the level 3 implementation of the level 1 relationship

```
-- Parent table
CREATE TABLE departments (
    department_id   NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    department_name VARCHAR2(100) NOT NULL,
    location        VARCHAR2(100)
);

-- Child table: holds the parent's primary key as a foreign key
CREATE TABLE employees (
    employee_id   NUMBER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    full_name     VARCHAR2(200) NOT NULL,
    department_id NUMBER        NOT NULL,
    hire_date     DATE          NOT NULL,
    CONSTRAINT fk_emp_dept
        FOREIGN KEY (department_id)
        REFERENCES departments (department_id)
);

-- Index on the foreign key column -- Oracle does NOT create this automatically
CREATE INDEX idx_emp_dept_id ON employees (department_id);
```

Relationships in Oracle

- Many-to-many
  - Requires a junction table
  - The many-to-many example realizes
    - "employees are assigned to projects"
  - The junction table is not in the domain model explicitly
    - Implied by the many-to-many relationship
  - Required at Level 3
    - Because the relational model has no direct way to express a many-to-many without it
  - Relationships can have their own attributes
    - assigned_date and role are attributes of the assignment relationship, not of the employee or the project

```
CREATE TABLE project_assignments (
    employee_id  NUMBER NOT NULL,
    project_id   NUMBER NOT NULL,
    assigned_date DATE  NOT NULL,
    role          VARCHAR2(50),
    CONSTRAINT pk_proj_assign  PRIMARY KEY (employee_id, project_id),
    CONSTRAINT fk_assign_emp   FOREIGN KEY (employee_id)  REFERENCES employees (employee_id),
    CONSTRAINT fk_assign_proj  FOREIGN KEY (project_id)   REFERENCES projects (project_id)
);

CREATE INDEX idx_assign_proj ON project_assignments (project_id);
```

Relationships in Oracle

- Foreign key constraint
  - Every non-null value in the foreign key column must exist as a primary or unique key value in the referenced table
  - Oracle checks this on every INSERT and UPDATE of the child table
  - Oracle checks this on every DELETE and UPDATE of the parent table that would leave orphaned child rows

ON DELETE clause	Behavior
(none -- the default)	Prevents deletion of a parent row that has child rows; raises ORA-02292
ON DELETE CASCADE	Automatically deletes all child rows when the parent is deleted
ON DELETE SET NULL	Sets the foreign key column to NULL in all child rows when the parent is deleted

Relationships in Oracle

- Foreign key constraint
  - Action on the child foreign key must be taken if the parent key is deleted
  - Default
    - Used when orphaned children represent a data integrity error
    - The application must explicitly handle parent deletion
  - CASCADE:
    - Use when children have no independent meaning without the parent
    - For example, order line items when an order is deleted

ON DELETE clause	Behavior
(none -- the default)	Prevents deletion of a parent row that has child rows; raises ORA-02292
ON DELETE CASCADE	Automatically deletes all child rows when the parent is deleted
ON DELETE SET NULL	Sets the foreign key column to NULL in all child rows when the parent is deleted

Relationships in Oracle

- Foreign key constraint
  - SET NULL
    - Use when the child can legitimately exist without a parent
    - For example, an employee whose department is dissolved

ON DELETE clause	Behavior
(none -- the default)	Prevents deletion of a parent row that has child rows; raises ORA-02292
ON DELETE CASCADE	Automatically deletes all child rows when the parent is deleted
ON DELETE SET NULL	Sets the foreign key column to NULL in all child rows when the parent is deleted

Relationships in Oracle

- Unindexed foreign key hazard
  - Oracle automatically indexes the referenced column (the primary key side)
  - Oracle does NOT automatically index the referencing column (the foreign key side)
  - When a parent row is deleted, Oracle must verify no child rows exist
  - Without an index on the child's foreign key column, Oracle performs a full table scan of the child table
    - Under concurrent load, this full scan requires a share lock on the child table, blocking all inserts and updates
    - This is one of the most common causes of unexpected lock contention in Oracle OLTP systems

ON DELETE clause	Behavior
(none -- the default)	Prevents deletion of a parent row that has child rows; raises ORA-02292
ON DELETE CASCADE	Automatically deletes all child rows when the parent is deleted
ON DELETE SET NULL	Sets the foreign key column to NULL in all child rows when the parent is deleted

Relationships in Oracle

- UPDATE constraints
  - Oracle does not provide an ON UPDATE CASCADE clause
  - Reflects Oracle's position on primary key design
    - A primary key that needs to change is a poorly chosen primary key
    - Surrogate keys generated by a sequence or GENERATED AS IDENTITY never need to change by definition
    - Natural keys that change are precisely the instability problem that surrogate keys exist to solve

ON UPDATE clause	Behavior	Use When	Risk
(none -- the default)	Prevents updating the primary key of a parent row that has child rows; raises ORA-02292	The parent primary key should never change -- which is always the case for a surrogate key and usually the correct design for a natural key	None -- this is the safe default and the reason surrogate keys are valuable: they never need to change
Application-managed cascade	The application explicitly updates child foreign key values before or after updating the parent key, within the same transaction	The natural key genuinely needs to change and referential integrity must be maintained	Requires the application to know every child table; easy to miss a table and leave orphaned rows
ON DELETE CASCADE combined with re-insert	Delete the parent row and re-insert it with the new key value, relying on ON DELETE CASCADE to remove children, then re-insert children with the new key	Rarely appropriate -- a workaround for the absence of ON UPDATE CASCADE in Oracle	High risk: delete and re-insert is not atomic across all child tables without careful transaction management; identity columns will generate a new surrogate value

Constraints

Constraints

- Constraints belong in the database
  - Not the application
- The fundamental argument
  - A database is not accessed exclusively through one application
  - Data is touched by multiple consumers
    - The main application, admin tools, ETL pipelines, reporting systems, migration scripts, bulk loaders, and direct SQL from operators
  - Business rules enforced only in application code are invisible to, and unenforced by, every other consumer
  - Business rules expressed as database constraints are enforced for every consumer, unconditionally

Constraint	What It Enforces	Oracle Mechanism
PRIMARY KEY	Each row is uniquely and non-nullably identified	Unique index, implicit NOT NULL
UNIQUE	A column or column combination is unique across all rows	Unique index, nulls permitted
NOT NULL	A column must have a value for every row	Column-level enforcement, no separate index
CHECK	A column value must satisfy a boolean expression	Evaluated on every INSERT and UPDATE
FOREIGN KEY	A column value must reference an existing parent row	Validated against parent table on DML

Constraints

- A constraint is a machine-readable business rule
  - Documents intent and enforces it simultaneously
  - A NOT NULL constraint says
    - "This attribute is mandatory for every instance of this entity"
  - A CHECK constraint says
    - "These are the only valid states this entity can be in"
  - A FOREIGN KEY say
    - "This entity cannot exist without a valid parent"
  - Removing a constraint removes a business rule from the schema permanently

Constraint	What It Enforces	Oracle Mechanism
PRIMARY KEY	Each row is uniquely and non-nullably identified	Unique index, implicit NOT NULL
UNIQUE	A column or column combination is unique across all rows	Unique index, nulls permitted
NOT NULL	A column must have a value for every row	Column-level enforcement, no separate index
CHECK	A column value must satisfy a boolean expression	Evaluated on every INSERT and UPDATE
FOREIGN KEY	A column value must reference an existing parent row	Validated against parent table on DML

Designing Constraints

- Always name constraints explicitly
  - Naming convention
    - pk_ for primary keys
    - fk_ for foreign keys
    - uq_ for unique,
    - hk_ for check
    - nn_ for not null (when named)
  - Include the table name and column(s) in the constraint name
  - Consistent naming makes error messages readable and scripts maintainable

```
-- BAD: Oracle assigns a system-generated name like SYS_C0047821
CREATE TABLE employees (
    employee_id NUMBER PRIMARY KEY,
    email       VARCHAR2(200) NOT NULL UNIQUE
);

-- GOOD: named constraints are identifiable in error messages and data dictionary
CREATE TABLE employees (
    employee_id NUMBER        NOT NULL,
    email       VARCHAR2(200) NOT NULL,
    CONSTRAINT pk_employees        PRIMARY KEY (employee_id),
    CONSTRAINT uq_employees_email  UNIQUE      (email),
    CONSTRAINT chk_emp_email_fmt   CHECK       (email LIKE '%@%')
);
```

Designing Constraints

- Deferrable constraints
  - By default, Oracle validates constraints at the end of each statement
  - DEFERRABLE INITIALLY DEFERRED delays validation until COMMIT
  - Useful for circular dependency insertion scenarios or bulk loads where intermediate states violate constraints
  - Use with care: deferred validation reduces the feedback loop for constraint violations

```
CONSTRAINT fk_emp_manager
    FOREIGN KEY (manager_id)
    REFERENCES employees (employee_id)
    DEFERRABLE INITIALLY DEFERRED
```

Designing Constraints

- The classic use case of DEFFEREABLE
  - A self-referencing table where the root row references itself
    - Or where two tables have circular foreign keys
  - Inserting data into such a structure requires temporarily violating a constraint in the middle of a transaction
  - Deferring validation to COMMIT allows the complete set of rows to be inserted before any constraint is checked

```
CONSTRAINT fk_emp_manager
    FOREIGN KEY (manager_id)
    REFERENCES employees (employee_id)
    DEFERRABLE INITIALLY DEFERRED
```

Designing Constraints

- Enabling and disabling constraints
  - Constraints are disabled for bulk loads
  - When Oracle enforces a foreign key constraint, it validates every row being inserted against the parent table
    - For a bulk load of millions of rows this becomes a significant overhead
    - Each row triggers a lookup against the parent table's index to confirm the referenced key exists
    - Disabling the constraint removes this per-row check entirely, allowing the bulk load to run at maximum speed

State	New rows checked	Existing rows checked	Typical use
ENABLE VALIDATE	Yes	Yes	Normal production state -- full enforcement
ENABLE NOVALIDATE	Yes	No	Turning a constraint back on after a bulk load when existing data is trusted but not yet verified
DISABLE NOVALIDATE	No	No	During bulk load -- no enforcement at all

Designing Constraints

- Enabling and disabling constraints
  - Disabling a constraint removes all enforcement
  - If the bulk load introduces rows with invalid foreign key values
    - For example, referencing parent rows that do not exist
    - Oracle accepts them silently
  - When the constraint is re-enabled with VALIDATE
    - Oracle finds those rows and raises an error, and the re-enable fails
    - The data must be cleaned before the constraint can be restored

State	New rows checked	Existing rows checked	Typical use
ENABLE VALIDATE	Yes	Yes	Normal production state -- full enforcement
ENABLE NOVALIDATE	Yes	No	Turning a constraint back on after a bulk load when existing data is trusted but not yet verified
DISABLE NOVALIDATE	No	No	During bulk load -- no enforcement at all

Designing Constraints

- The three states a constraint can be in
  - A constraint in Oracle is has two independent properties
  - Enabled or Disabled
    - Whether Oracle enforces the rule for new DML operations (inserts and updates)
  - Validated or Not Validated
    - Whether Oracle has confirmed that all existing rows in the table satisfy the rule
  - These combine into three meaningful states shown in the table
    - The fourth state DISABLE VALIDATE has almost no practical use

State	New rows checked	Existing rows checked	Typical use
ENABLE VALIDATE	Yes	Yes	Normal production state -- full enforcement
ENABLE NOVALIDATE	Yes	No	Turning a constraint back on after a bulk load when existing data is trusted but not yet verified
DISABLE NOVALIDATE	No	No	During bulk load -- no enforcement at all

Designing Constraints

- The correct sequence for a bulk load
  - Combining ENABLE VALIDATE in a single statement on a large table
    - Oracle must scan every row before it will accept the command
    - If a single invalid row exists the entire operation fails and the constraint remains disabled.
  - By separating NOVALIDATE and VALIDATE
    - New DML is protected immediately while the validation scan runs as a separate, explicit operation

```
-- Step 1: disable the constraint before the load
-- existing rows were already valid; new rows will be checked after
ALTER TABLE employees DISABLE CONSTRAINT fk_emp_dept;

-- Step 2: run the bulk load
INSERT /*+ APPEND */ INTO employees ...

-- Step 3: re-enable with NOVALIDATE first
-- this turns enforcement back on for future DML immediately
-- without paying the cost of scanning every existing row yet
ALTER TABLE employees ENABLE NOVALIDATE CONSTRAINT fk_emp_dept;

-- Step 4: validate existing rows as a separate, explicit step
-- this scans the entire table and fails if any row violates the constraint
-- if it fails, the invalid rows can be identified and cleaned before retrying
ALTER TABLE employees ENABLE VALIDATE CONSTRAINT fk_emp_dept;
```

Designing Constraints

- The correct sequence for a bulk load
  - If the validation fails
    - The constraint is in ENABLE NOVALIDATE state ensures new rows are being checked
    - The invalid existing rows can be identified using Oracle's EXCEPTIONS INTO mechanism before retrying the full validation.

```
-- Identifying rows that violate the constraint before attempting ENABLE VALIDATE
CREATE TABLE constraint_exceptions (
    row_id       UROWID,
    owner        VARCHAR2(128),
    table_name   VARCHAR2(128),
    constraint   VARCHAR2(128)
);

ALTER TABLE employees
    ENABLE VALIDATE CONSTRAINT fk_emp_dept
    EXCEPTIONS INTO constraint_exceptions;

-- Rows that violated the constraint are now listed in constraint_exceptions
-- with their UROWID, allowing them to be identified, corrected, and retried
SELECT e.*
FROM   employees e
JOIN   constraint_exceptions ce ON ce.row_id = e.rowid;
```

Questions?